Loan Eligibility with Ensemble Boosting

Table of Contents

1. Abstract

Banks and all sort of credit institutes which provide *loans*, face great challenges when a new potential customer asks for a loan.
These challenges translate into *risks*, which if not properly balanced, may end up with catastrophic consequences for all clients in the financial-chain, and may have a big impact in the outside world too.
Technology can mitigate these risks.

By gathering a huge amout of *data*, and by applying some analytics we can see the *hidden shape* of the risks that credit institutes take.
Then, thanks to some state-of-the-art algorithms we can predict the potential risk that a new client may represent for a bank.

At the end of the day, the aim is to provide a robust model to support difficult *decision-making* scenarios.

The purpose of the notebook is to show and manipulate a huge dataset collected from Kaggle, then after some *feature engineering* we apply and compare three State-of-the-Art *ensemble learning* algorithms for tabular data:

- AdaBoost
- GradientBoost
- XGBoost

Based on statistics we decide which the best one is.
Further conclusions and possible expansions are presented.

pointer to the dataset on Kaggle

See Appendix 7.B. to understand how this image has been generated

2. Dataset Import

In [1]:
```
#import of all libraries and packages
import kagglehub #for automatically downloading the dataset from Kaggle IF it has c
import pandas as pd #data manipulation
import numpy as np #numerical manipulation
import matplotlib.pyplot as plt #data visualization
import matplotlib.patches as mpatches #for distinguishing the dots in the scatter m
import sklearn #ML
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder #handling categoric
import sys #get the real size of the DataFrames
```

In [2]:
```
#Code to fetch the data
try:
    #Download latest version
    path = kagglehub.dataset_download("yasserh/loan-default-dataset")

    #to check where the dataset has been located
```

```
    #print("Path to dataset files:", path)

    loan_data = pd.read_csv(path + '\Loan_Default.csv')

#if something goes wrong (e.g.: connection, wrong path, ...), use the copy of the d
except:
    loan_data = pd.read_csv('Loan_Default.csv')
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.1
2), please consider upgrading to the latest version (0.3.13).
```

2.1. Dataset Description

In [3]:
```
pd.set_option('display.max_columns',
              None) #display all columns
loan_data.head()
```

Out[3]:

	ID	year	loan_limit	Gender	approv_in_adv	loan_type	loan_purpose	Credit_Wortl
0	24890	2019	cf	Sex Not Available	nopre	type1	p1	
1	24891	2019	cf	Male	nopre	type2	p1	
2	24892	2019	cf	Male	pre	type1	p1	
3	24893	2019	cf	Male	nopre	type1	p4	
4	24894	2019	cf	Joint	pre	type1	p1	

This description may be very helpful through all the notebook's reading, so keep it in sight.

- **ID:** client loan application id
- **year:** year of loan application
- **loan limit:** indicates whether the loan is conforming (cf) or non-conforming (ncf)
- **Gender:** gender of the applicant (male, female, joint, sex not available)
- **approv_in_adv:** indicates whether the loan was approved in advance (pre, nopre)
- **loan_type:** type of loan (type1, type2, type3) :
  - *Type 1 (Conventional Loans):* Characterized by higher loan amounts, lower LTV ratios, and stronger credit scores, making them a preferred option for well-qualified, lower-risk borrowers.
  - *Type 2 (Government-Backed Loans):* Typically involve lower loan amounts, higher LTV ratios, and moderate credit scores, indicating they are used by borrowers with smaller down payments who benefit from government-backed programs.
  - *Type 3 (Non-Conventional Loans):* Feature moderate loan amounts, the highest LTV ratios, and lower credit scores, often associated with higher-risk products such as

jumbo loans or adjustable-rate mortgages.

- **loan_purpose:** purpose of the loan (p1, p2, p3, p4):
    - *p1 (Home Purchase):* Represents loans taken out for primary residences, often displaying moderate credit scores and higher LTV ratios.
    - *p2 (Home Improvement):* Smaller loan amounts used for property renovations, with lower LTV ratios suggesting homeowners are leveraging built-up equity.
    - *p3 (Refinancing):* Applies to homeowners replacing an existing mortgage, characterized by moderate loan amounts and lower LTV ratios, indicating financial stability.
    - *p4 (Investment Property):* Involves larger loan amounts and higher risk profiles, primarily financed through conventional loans due to restrictions on Government-backed funding for investment properties.
- **Credit_Worthiness:** credit worthiness (l1, l2)
- **open_credit:** indicates whether the applicant has any open credit accounts (opc, nopc)
- **business_or_commercial:** indicates whether the loan is for business/commercial purposes (ob/c - business/commercial, nob/c - personal)
- **loan_amount:** amount of money being borrowed
- **rate_of_interest:** interest rate charged on the loan
- **Interest_rate_spread:** difference between the interest rate on the loan and a benchmark interest rate
- **Upfront_charges:** initial charges associated with securing the loan
- **term:** duration of the loan in months
- **Neg_ammortization:** indicates whether the loan allows for negative amortization (neg_amm, not_neg)
- **interest_only:** indicates whether the loan has an interest-only payment option (int_only, not_int)
- **lump_sum_payment:** indicates if a lump sum payment is required at the end of the loan term (lpsm, not_lpsm)
- **property_value:** value of the property being financed
- **construction_type:** type of construction (sb - site built, mh - manufactured home)
- **occupancy_type:** occupancy type (pr - primary residence, sr - secondary residence, ir - investment property)
- **Secured_by:** specifies the type of collateral securing the loan (home, land)
- **total_units:** number of units in the property being financed (1U, 2U, 3U, 4U)
- **income** [*] **:** applicant's annual income
- **credit_type:** applicant's type of credit (CIB - credit information bureau, CRIF - CIRF credit information bureau, EXP - experian, EQUI - equifax)
- **Credit_Score:** applicant's credit score
- **co-applicant_credit_type:** co-applicant's type of credit (CIB - credit information bureau, EXP - experian)
- **age:** the age of the applicant

- **submission_of_application:** indicates how the application was submitted (to_inst - to institution, not_inst - not to institution)
- **LTV:** loan-to-value ratio, calculated as the loan amount divided by the property value
- **Region:** geographic region where the property is located (North, South, Central, North-East)
- **Security_Type:** type of security or collateral backing the loan (direct, indirect)
- **Status:** indicates whether the loan has been defaulted (1) or not (0)
- **dtir1:** debt-to-income ratio

* The annual income is in thousands of dollars.

In [4]:
```
loan_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148670 entries, 0 to 148669
Data columns (total 34 columns):
 #   Column                     Non-Null Count   Dtype  
---  ------                     --------------   -----  
 0   ID                         148670 non-null  int64  
 1   year                       148670 non-null  int64  
 2   loan_limit                 145326 non-null  object 
 3   Gender                     148670 non-null  object 
 4   approv_in_adv              147762 non-null  object 
 5   loan_type                  148670 non-null  object 
 6   loan_purpose               148536 non-null  object 
 7   Credit_Worthiness          148670 non-null  object 
 8   open_credit                148670 non-null  object 
 9   business_or_commercial     148670 non-null  object 
 10  loan_amount                148670 non-null  int64  
 11  rate_of_interest           112231 non-null  float64
 12  Interest_rate_spread       112031 non-null  float64
 13  Upfront_charges            109028 non-null  float64
 14  term                       148629 non-null  float64
 15  Neg_ammortization          148549 non-null  object 
 16  interest_only              148670 non-null  object 
 17  lump_sum_payment           148670 non-null  object 
 18  property_value             133572 non-null  float64
 19  construction_type          148670 non-null  object 
 20  occupancy_type             148670 non-null  object 
 21  Secured_by                 148670 non-null  object 
 22  total_units                148670 non-null  object 
 23  income                     139520 non-null  float64
 24  credit_type                148670 non-null  object 
 25  Credit_Score               148670 non-null  int64  
 26  co-applicant_credit_type   148670 non-null  object 
 27  age                        148470 non-null  object 
 28  submission_of_application  148470 non-null  object 
 29  LTV                        133572 non-null  float64
 30  Region                     148670 non-null  object 
 31  Security_Type              148670 non-null  object 
 32  Status                     148670 non-null  int64  
 33  dtir1                      124549 non-null  float64
dtypes: float64(8), int64(5), object(21)
memory usage: 38.6+ MB
```

Notice: above the `info()` command, gives us a memory usage size that is actually a bit misleading, GitHub issue DataFrame.memory_usage.
That is the reason why we decided to use: `sys.getsizeof()` , since this data will be quite interesting especially for the next sections.

In [5]:
```
sys.getsizeof(loan_data) #returns the size in bytes
```

Out[5]:
```
207281850
```

In [6]:
```
loan_data.describe()
```

Out[6]:

	ID	year	loan_amount	rate_of_interest	Interest_rate_spread	Upfron
count	148670.000000	148670.0	1.486700e+05	112231.000000	112031.000000	1090
mean	99224.500000	2019.0	3.311177e+05	4.045476	0.441656	32
std	42917.476598	0.0	1.839093e+05	0.561391	0.513043	32
min	24890.000000	2019.0	1.650000e+04	0.000000	-3.638000	
25%	62057.250000	2019.0	1.965000e+05	3.625000	0.076000	5
50%	99224.500000	2019.0	2.965000e+05	3.990000	0.390400	25
75%	136391.750000	2019.0	4.365000e+05	4.375000	0.775400	48
max	173559.000000	2019.0	3.576500e+06	8.000000	3.357000	600

Since the columns: `ID` and `year` are not meaningful for any purpose, we can drop them from the very beginning and still be compliant to the best preprocessing practices.

In [7]:

```
#since they are not meaningful columns at all, we can drop them from the whole data
loan_data = loan_data.drop('ID', axis=1)
loan_data = loan_data.drop('year', axis=1)
```

In [8]:

```
#to get insights on the categorical data
categorical_values_counts = list()
for cat in loan_data.select_dtypes(include=['object']).columns: #extract the list o
    categorical_values_counts.append(loan_data[f"{cat}"].value_counts())
    categorical_values_counts.append('-'*45) #for spacing them
categorical_values_counts
```

```
Out[8]: [cf     135348
 ncf      9978
 Name: loan_limit, dtype: int64,
 '-----------------------------------------------------',
 Male                 42346
 Joint                41399
 Sex Not Available    37659
 Female               27266
 Name: Gender, dtype: int64,
 '-----------------------------------------------------',
 nopre    124621
 pre       23141
 Name: approv_in_adv, dtype: int64,
 '-----------------------------------------------------',
 type1    113173
 type2     20762
 type3     14735
 Name: loan_type, dtype: int64,
 '-----------------------------------------------------',
 p3    55934
 p4    54799
 p1    34529
 p2     3274
 Name: loan_purpose, dtype: int64,
 '-----------------------------------------------------',
 l1    142344
 l2      6326
 Name: Credit_Worthiness, dtype: int64,
 '-----------------------------------------------------',
 nopc    148114
 opc        556
 Name: open_credit, dtype: int64,
 '-----------------------------------------------------',
 nob/c    127908
 b/c       20762
 Name: business_or_commercial, dtype: int64,
 '-----------------------------------------------------',
 not_neg    133420
 neg_amm     15129
 Name: Neg_ammortization, dtype: int64,
 '-----------------------------------------------------',
 not_int     141560
 int_only      7110
 Name: interest_only, dtype: int64,
 '-----------------------------------------------------',
 not_lpsm    145286
 lpsm          3384
 Name: lump_sum_payment, dtype: int64,
 '-----------------------------------------------------',
 sb    148637
 mh        33
 Name: construction_type, dtype: int64,
 '-----------------------------------------------------',
 pr    138201
 ir      7340
 sr      3129
```

```
Name: occupancy_type, dtype: int64,
'----------------------------------------------',
home    148637
land        33
Name: Secured_by, dtype: int64,
'----------------------------------------------',
1U    146480
2U      1477
3U       393
4U       320
Name: total_units, dtype: int64,
'----------------------------------------------',
CIB     48152
CRIF    43901
EXP     41319
EQUI    15298
Name: credit_type, dtype: int64,
'----------------------------------------------',
CIB    74392
EXP    74278
Name: co-applicant_credit_type, dtype: int64,
'----------------------------------------------',
45-54    34720
35-44    32818
55-64    32534
65-74    20744
25-34    19142
>74       7175
<25       1337
Name: age, dtype: int64,
'----------------------------------------------',
to_inst     95814
not_inst    52656
Name: submission_of_application, dtype: int64,
'----------------------------------------------',
North         74722
south         64016
central        8697
North-East     1235
Name: Region, dtype: int64,
'----------------------------------------------',
direct      148637
Indriect        33
Name: Security_Type, dtype: int64,
'----------------------------------------------']
```

2.2. Missing Values

We can see that the biggest issues in terms of missing values come from: `Upfront_charges` , `Interest_rate_spread` , `rate_of_interest` . we could have some criticalities also on other features but still negligible, even though we have to keep in mind that the `NaN` are spread out as:

Numerical

- 39642 Upfront_charges
- 36639 Interest_rate_spread
- 36439 rate_of_interest
- 24121 dtir1
- 15098 LTV
- 15098 property_value
- 9150 income
- 41 term

Categorical

- 3344 loan_limit
- 908 approv_in_adv
- 200 age
- 200 submission_of_application
- 136 loan_purpose
- 121 Neg_ammortization

In the next code blocks we will see how to approach this problem.

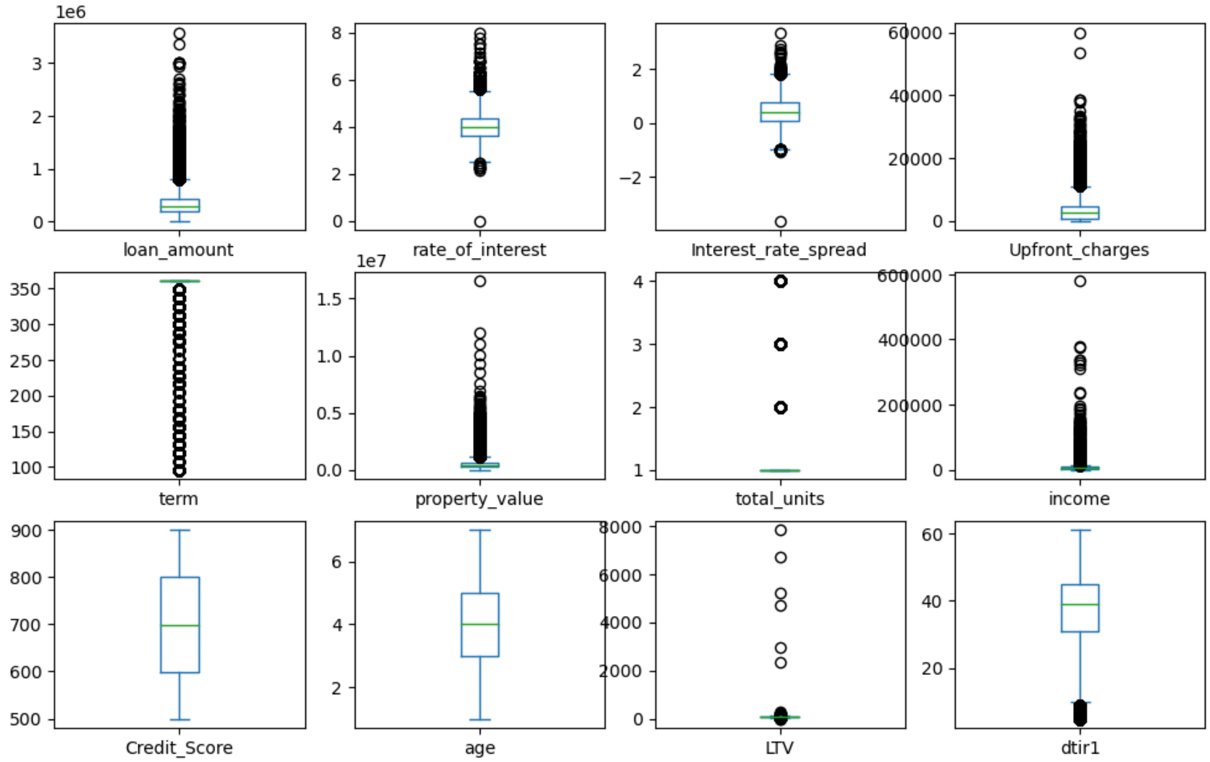
2.3. Numerical and Ordinal Data

In [9]:
```
#converting 'total_units' and 'age' categorical data into ordinal
ordinal_encoder = OrdinalEncoder()

#just for the total_units and age features, since it makes sense a concept of ordin
total_units_cat = loan_data[['total_units']] #take the column we want to encode
total_units_encoded = ordinal_encoder.fit_transform(total_units_cat) + 1 #we shift
loan_data['total_units'] = total_units_encoded #replace the column in the dataset

age_mapping = {
    '<25':1,
    '25-34':2,
    '35-44':3,
    '45-54':4,
    '55-64':5,
    '65-74':6,
    '>74':7
}
loan_data['age'] = loan_data['age'].map(age_mapping) #replace the column in the dat
```

In [10]:
```
#list containing all the numerical + ordinal features plus the target
numerical_ordinal_columns = [col for col in loan_data.columns if col not in loan_da
#list with only the numerical + ordinal features
numerical_ordinal_features = numerical_ordinal_columns.copy()
numerical_ordinal_features.remove('Status')
```

In [11]:
```
#boxplots
loan_data[numerical_ordinal_features].plot(kind='box',
```

```
                                                     subplots=True, #distinct box plots for each feature
                                                     layout= (int(len(numerical_ordinal_features) ** 0.5)
                                                     figsize=((int(len(numerical_ordinal_features) ** 0.5
                                                     sharex=False, #each subplot uses a different x scale
                                                     sharey=False #each subplot uses a different y scale
                                                    )
plt.show()
```



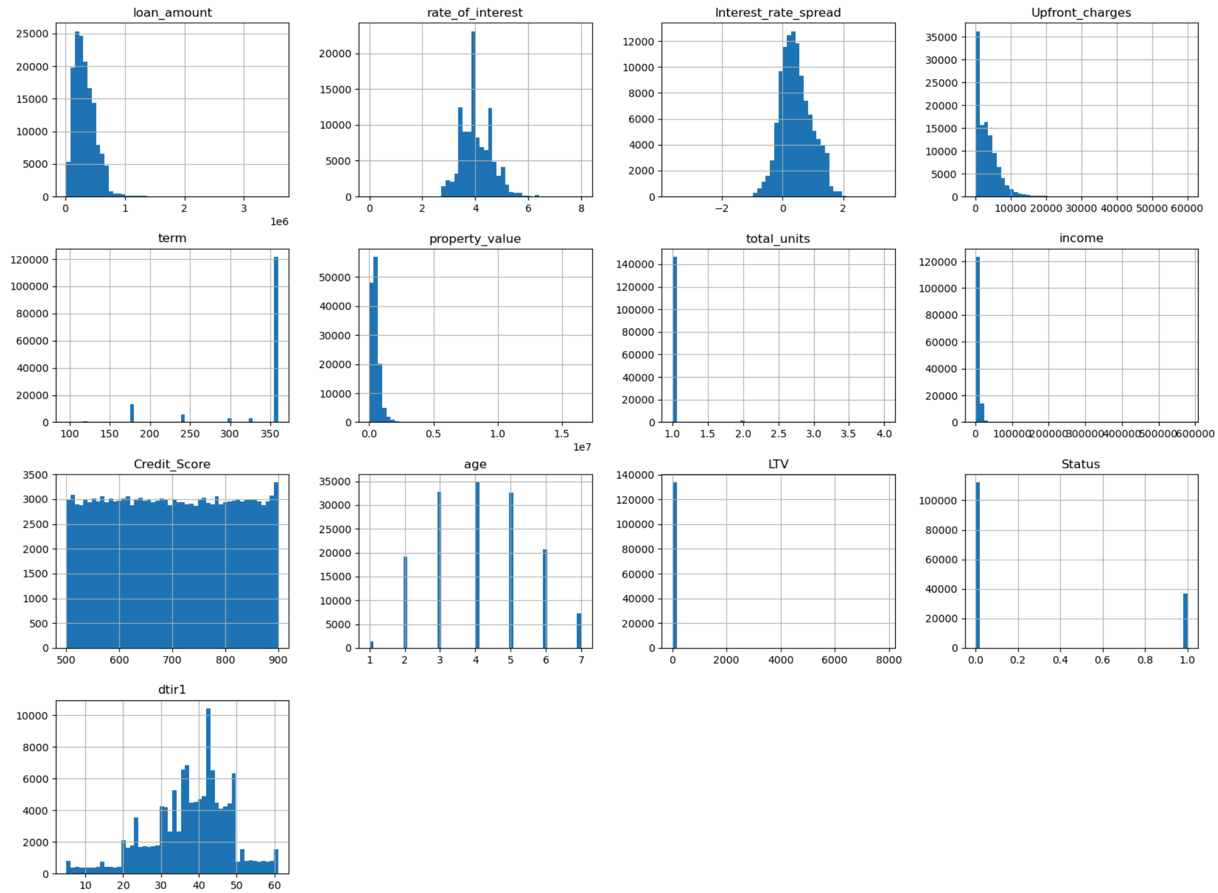
The box plots above show that in several occasions we encounter outliers.
Remember from statistics that:

$$ \text{Outlier Range} = [\text{Q}_1 - 1.5 \cdot \text{IQR}\text{;} \ \text{Q}_3 + 1.5 \cdot \text{IQR}] $$

Where:

- Q_1: first quartile
- Q_3: third quartile
- InterQuartile Range: $ \text{IQR} = \text{Q}_3 - \text{Q}_1 $

In [12]:
```
#plot the histograms
loan_data.hist(bins=50, #number of bins to encapsulate the data
                figsize=(20,15)
               );
```



Notice: There are some capped values, specifically concerning:

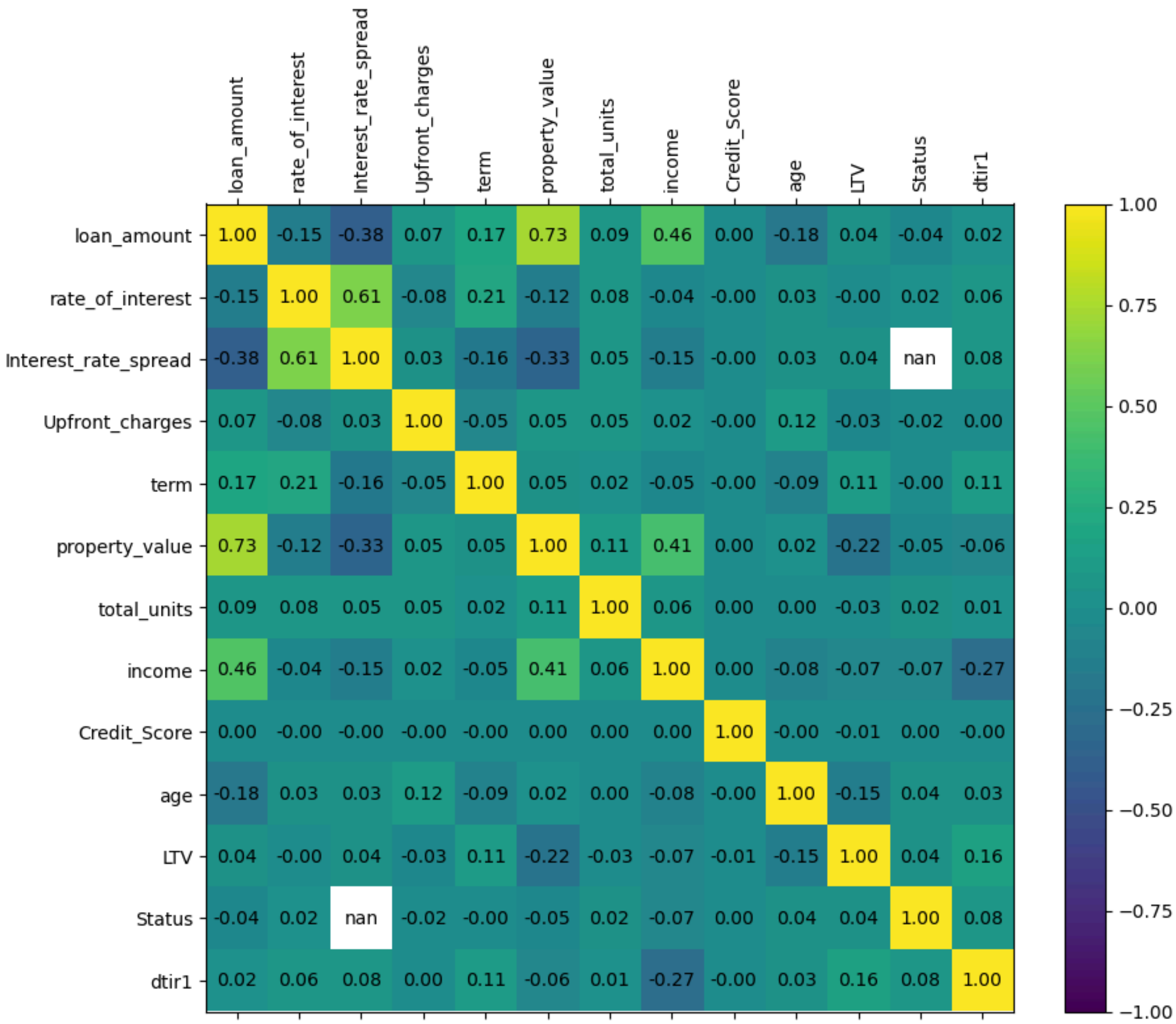
- term
- dtir1
- Credit_Score

`term` feature for sure is capped at: 360. Probably also: `dtir1` and `Credit_Sore`, were capped but with a lower impact.
In any case, since they are not our target attributes, this is not a big issue and we can actually ignore it.

In [13]:
```
loan_data.corr()["Status"].sort_values(ascending=False) #correlation related to out
```

Out[13]:
```
Status                  1.000000
dtir1                   0.078083
age                     0.044600
LTV                     0.038895
total_units             0.023800
rate_of_interest        0.022957
Credit_Score            0.004004
term                   -0.000240
Upfront_charges        -0.019138
loan_amount            -0.036825
property_value         -0.048864
income                 -0.065119
Interest_rate_spread         NaN
Name: Status, dtype: float64
```

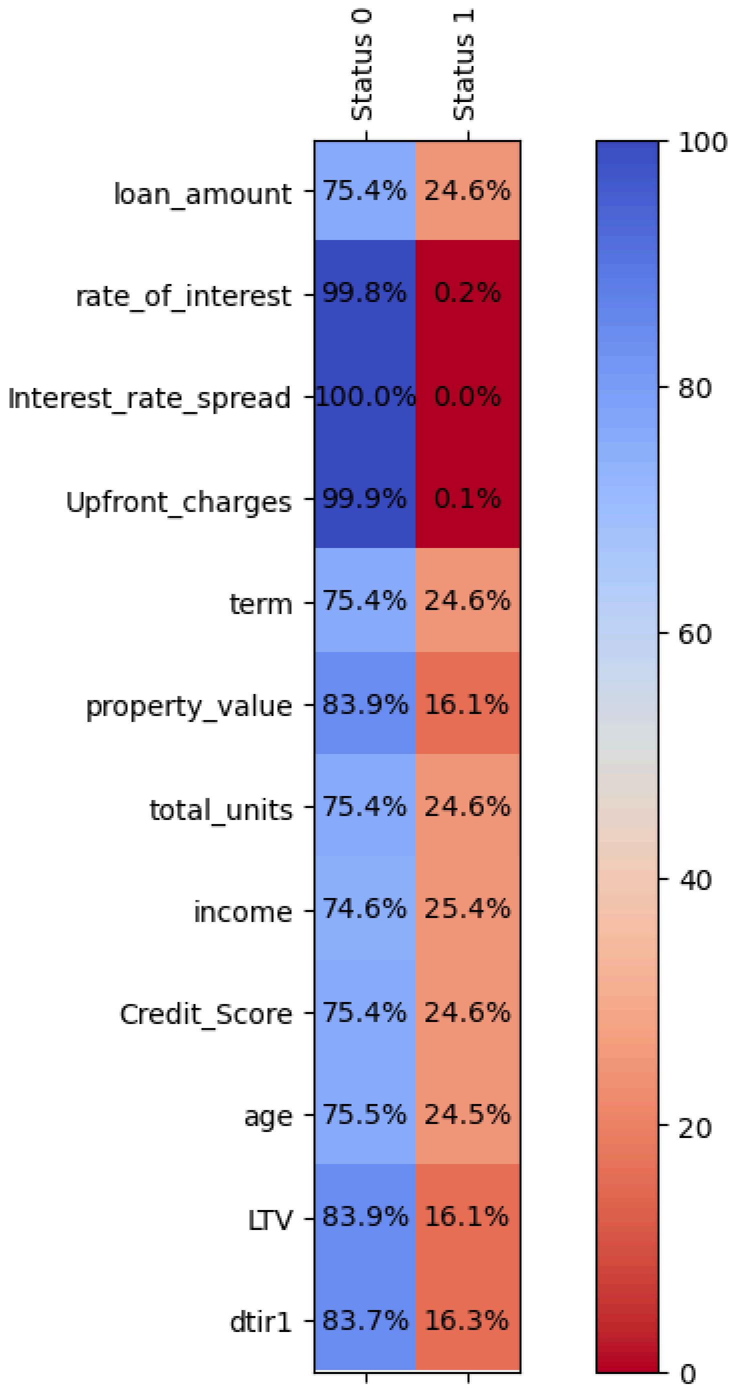
In [14]:
```python
#compute the correlations
correlations = loan_data.corr()

#plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111)
cax = ax.matshow(correlations,
                 vmin=-1,
                 vmax=1)
fig.colorbar(cax)
ticks = np.arange(len(numerical_ordinal_columns))
ax.set_xticks(ticks)
ax.set_yticks(ticks)
ax.set_xticklabels(numerical_ordinal_columns,
                   rotation=90) #to see the column lables clearly
ax.set_yticklabels(numerical_ordinal_columns)

# Annotate cells with correlation values
for i in range(len(correlations)):
    for j in range(len(correlations)):
        value = correlations.iloc[i, j]
        ax.text(j,
                i,
                f'{value:.2f}',
                va='center',
                ha='center',
                color='black')

plt.show()
```



Notice: the correlation coefficient between `Status` and `interest_rate_spread` returns not a number, this is due to the fact that the latter feature is actually compromised, (we will see in few code blocks how), furthermore the two standard deviations are very tiny.

In [15]:
```
#showing the percentages of data grouped by target values
def compute_feature_target_percentages(df, features, target='Status'):
    rows = []
    for feat in features:
        #Discharge NaN values for feature and target
        feature_data = df[[feat, target]].dropna()

        #total valid values for this feature
        total = len(feature_data)

        #How many of these are target=0 and target=1
        count_0 = feature_data[feature_data[target] == 0].shape[0]
        count_1 = feature_data[feature_data[target] == 1].shape[0]

        #we translate the above data into percentages
        pct_0 = (count_0 / total) * 100 if total > 0 else 0
        pct_1 = (count_1 / total) * 100 if total > 0 else 0
```

```python
        rows.append({'feature': feat,
                     'target=0 %': pct_0,
                     'target=1 %': pct_1})
    return pd.DataFrame(rows)

result = compute_feature_target_percentages(loan_data, numerical_ordinal_features)

#Extract data and labels
data_matrix = result[['target=0 %',
                      'target=1 %']].to_numpy()
feature_labels = result['feature'].tolist()
target_labels = ['Status 0',
                 'Status 1']

#Plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111)
cax = ax.matshow(data_matrix,
                 vmin=0,
                 vmax=100,
                 cmap='coolwarm_r')
fig.colorbar(cax)

#Ticks
ax.set_xticks(np.arange(len(target_labels)))
ax.set_yticks(np.arange(len(feature_labels)))
ax.set_xticklabels(target_labels,
                   rotation=90)
ax.set_yticklabels(feature_labels)

#Values
for i in range(data_matrix.shape[0]):
    for j in range(data_matrix.shape[1]):
        value = data_matrix[i, j]
        ax.text(j,
                i, f'{value:.1f}%',
                va='center',
                ha='center',
                color='black')
plt.show()
```



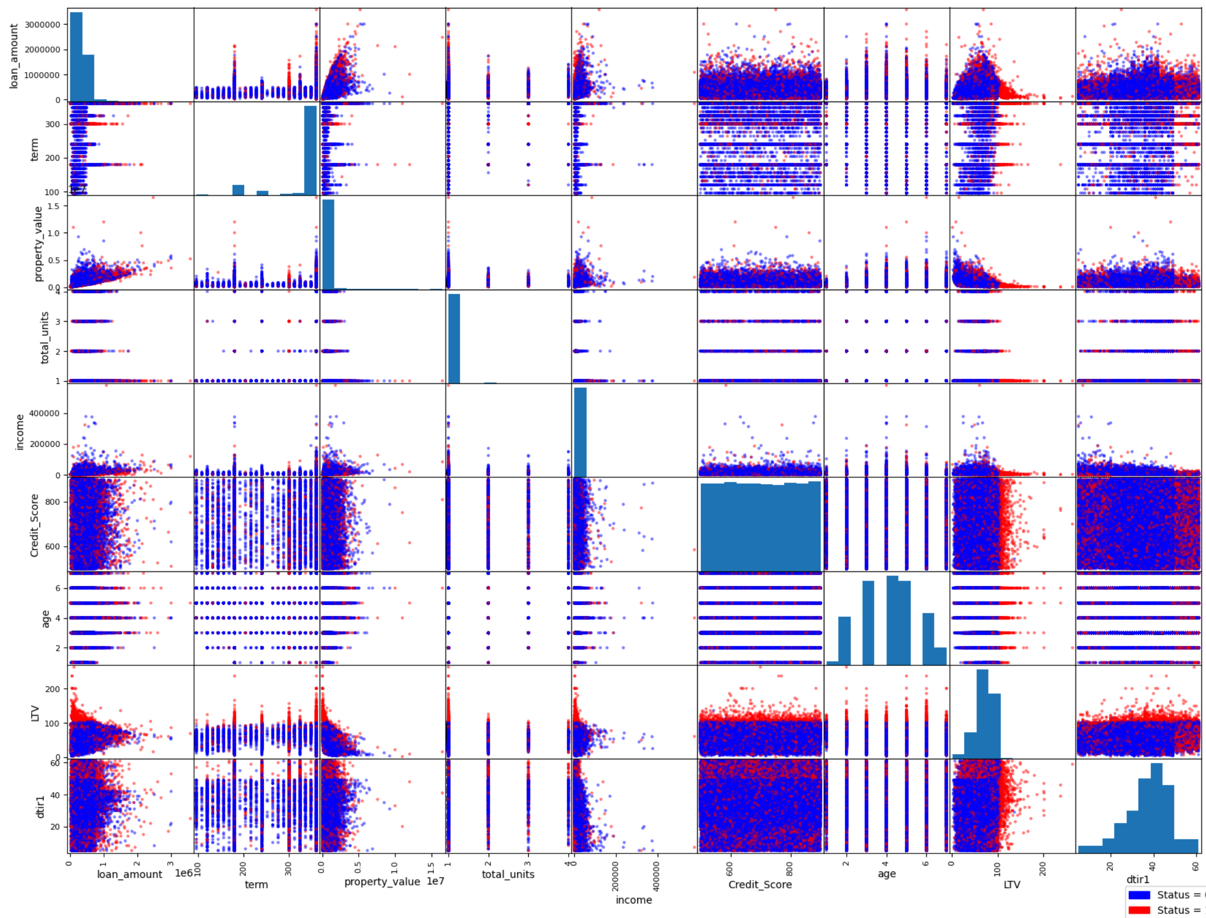
Notice: it is important to point out that through the notebook we adopted the following convention:

- `Status=0` , *i.e.:* loan totally repaid, is chosen as the ***negative*** case
- `Status=1` , *i.e.:* loan not repaid, is chosen as the ***positive*** case

This is due to the general rule that the *positive* class should be the one of interest, so the event that we want to detect, indeed for us it is important to detect the default client.

Due to what we saw in section: 2.2. Missing Values, we can say that: `rate_of_interest` , `Interest_rate_spread` and `Upfront_charges` , when they are not missing they are nearly always with `Status=0`. We must drop them entirely from the whole dataset, because they are not meaningful and they have also the highest misses.

In [16]:
```
#discharge the not desired columns
data = loan_data.drop(['rate_of_interest',
                       'Interest_rate_spread',
                       'Upfront_charges'],
                      axis=1)

#discharge the rows without elements
data = data.dropna()

#prepare the colors based on the Status (0 o 1)
color_map = {0: 'blue',
             1: 'red'
            }
colors = data['Status'].map(color_map)

# Plot of the scatter_matrix
pd.plotting.scatter_matrix(data.drop(columns=['Status']),
                            figsize=(20, 15),
                            diagonal='hist',
                            color=colors,
                            alpha=0.5
            )

#patch for the legend
legend_handles = [mpatches.Patch(color='blue',
                                label='Status = 0'
                               ),
                  mpatches.Patch(color='red',
                                label='Status = 1'
                               )
                 ]

plt.legend(handles=legend_handles,
           loc='upper right',
           bbox_to_anchor=(1.15, -0.3)
          )
plt.show()
```



Notice: grouping dots based on the the target condition is very helpful for data visualization, there is a Python library for data visualization Seaborn * that allows to plot directly these types of graphs in a very straightforward way, but it is a more computational expensive tool, that in cases of big datasets can slow down the process a lot, this is the reason why we implemented the solution using `matplotlib` in a longer way.

In some cells of the matrix, the color division makes us able to draw a straight line that can divide the default and non-default loans in an approximately sharp manner.

*: Here is a project of mine with an implention using Seaborn

General Categorical and Binary Categorical Data

Notice: in the following block of code, we decided to transform manually the binary categorical data in binary ordinal data, assigning \$0\$ or \$1\$ with the following concept: \$0\$ whether the associated variable has lower probability of deafult, \$1\$ viceversa. (where possible).

In [17]:
```
#convert all the categorical binary columns to ordinal binary
loan_data['loan_limit'] = loan_data['loan_limit'].map({'cf': 1,'ncf': 0})
loan_data['approv_in_adv'] = loan_data['approv_in_adv'].map({'nopre': 1,'pre': 0})
loan_data['Credit_Worthiness'] = loan_data['Credit_Worthiness'].map({'l2': 1,'l1':
loan_data['open_credit'] = loan_data['open_credit'].map({'opc': 1,'nopc': 0})
```

```
loan_data['business_or_commercial'] = loan_data['business_or_commercial'].map({'b/c
loan_data['Neg_ammortization'] = loan_data['Neg_ammortization'].map({'not_neg': 1,'
loan_data['interest_only'] = loan_data['interest_only'].map({'not_int': 1,'int_only
loan_data['lump_sum_payment'] = loan_data['lump_sum_payment'].map({'not_lpsm': 1,'l
loan_data['construction_type'] = loan_data['construction_type'].map({'sb': 1,'mh':
loan_data['Secured_by'] = loan_data['Secured_by'].map({'home': 1,'land': 0})
loan_data['co-applicant_credit_type'] = loan_data['co-applicant_credit_type'].map({
loan_data['submission_of_application'] = loan_data['submission_of_application'].map
loan_data['Security_Type'] = loan_data['Security_Type'].map({'direct': 1,'Indriect'
```

In [18]:

```
#list of categorical non-binary columns
cat_columns = ['Gender',
               'loan_type',
               'loan_purpose',
               'occupancy_type',
               'credit_type',
               'Region']

# Loop through each column
for col in cat_columns:
    encoder = OneHotEncoder(sparse=False,
                            handle_unknown='ignore')  #new encoder for each
    encoded_array = encoder.fit_transform(loan_data[[col]])
    encoded_df = pd.DataFrame(encoded_array,
                              columns=encoder.get_feature_names_out([col]),
                              index=loan_data.index)

    # Drop original column and concatenate the encoded DataFrame
    loan_data.drop(columns=[col],
                   inplace=True)
    loan_data = pd.concat([loan_data,
                           encoded_df
                          ],
                          axis=1)
```

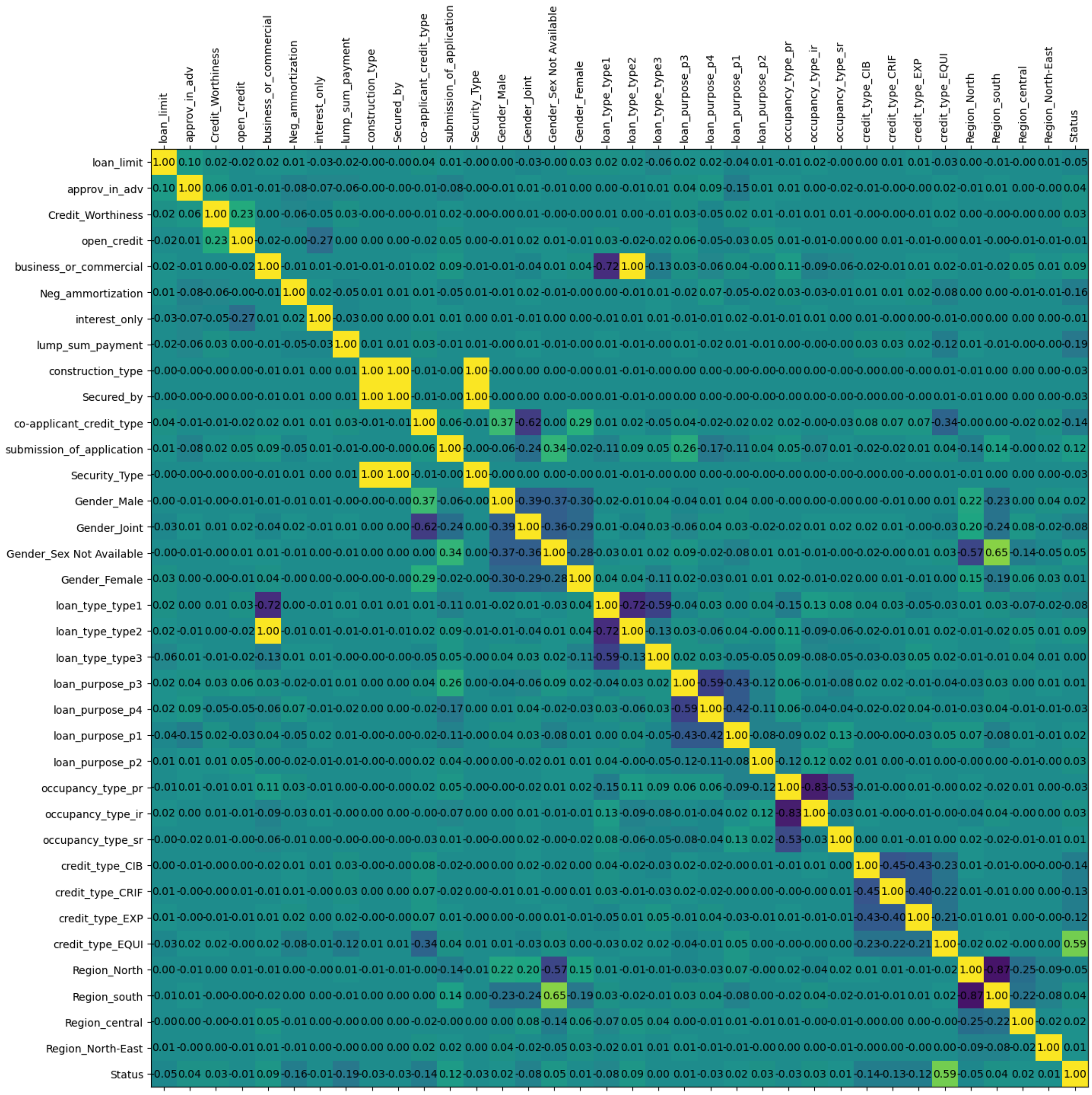
In [19]:

```
#list of all categorical columns
categorical_columns = [
                      'loan_limit',
                      'approv_in_adv',
                      'Credit_Worthiness',
                      'open_credit',
                      'business_or_commercial',
                      'Neg_ammortization',
                      'interest_only',
                      'lump_sum_payment',
                      'construction_type',
                      'Secured_by',
                      'co-applicant_credit_type',
                      'submission_of_application',
                      'Security_Type',
                      'Gender_Male', 'Gender_Joint', 'Gender_Sex Not Available', 'G
                      'loan_type_type1', 'loan_type_type2', 'loan_type_type3',
                      'loan_purpose_p3', 'loan_purpose_p4', 'loan_purpose_p1', 'loa
                      'occupancy_type_pr', 'occupancy_type_ir', 'occupancy_type_sr'
                      'credit_type_CIB', 'credit_type_CRIF', 'credit_type_EXP', 'cr
```

```
                       'Region_North', 'Region_south', 'Region_central', 'Region_Nor
                       'Status'
                     ]


categorical_features = categorical_columns.copy()
#list of all categorical features
categorical_features.remove('Status')
```

In [20]:

```
binary_correlation = loan_data[categorical_columns].corr()

#plot
fig = plt.figure(figsize=(20, 16))
ax = fig.add_subplot(111)
cax = ax.matshow(binary_correlation,
                 vmin=-1,
                 vmax=1)

ticks = np.arange(len(categorical_columns))
ax.set_xticks(ticks)
ax.set_yticks(ticks)
ax.set_xticklabels(categorical_columns,
                   rotation=90) #to see the column lables clearly
ax.set_yticklabels(categorical_columns)

#annotate cells with correlation values
for i in range(len(binary_correlation)):
    for j in range(len(binary_correlation)):
        value = binary_correlation.iloc[i, j]
        ax.text(j,
                i,
                f'{value:.2f}',
                va='center',
                ha='center',
                color='black')

plt.show()
```



Pandas `dataframe.corr()` method computes the Pearson's correlation coefficient:

$$ \rho_{x,y} = \frac{cov(X,Y)}{\sigma_X\sigma_Y} $$

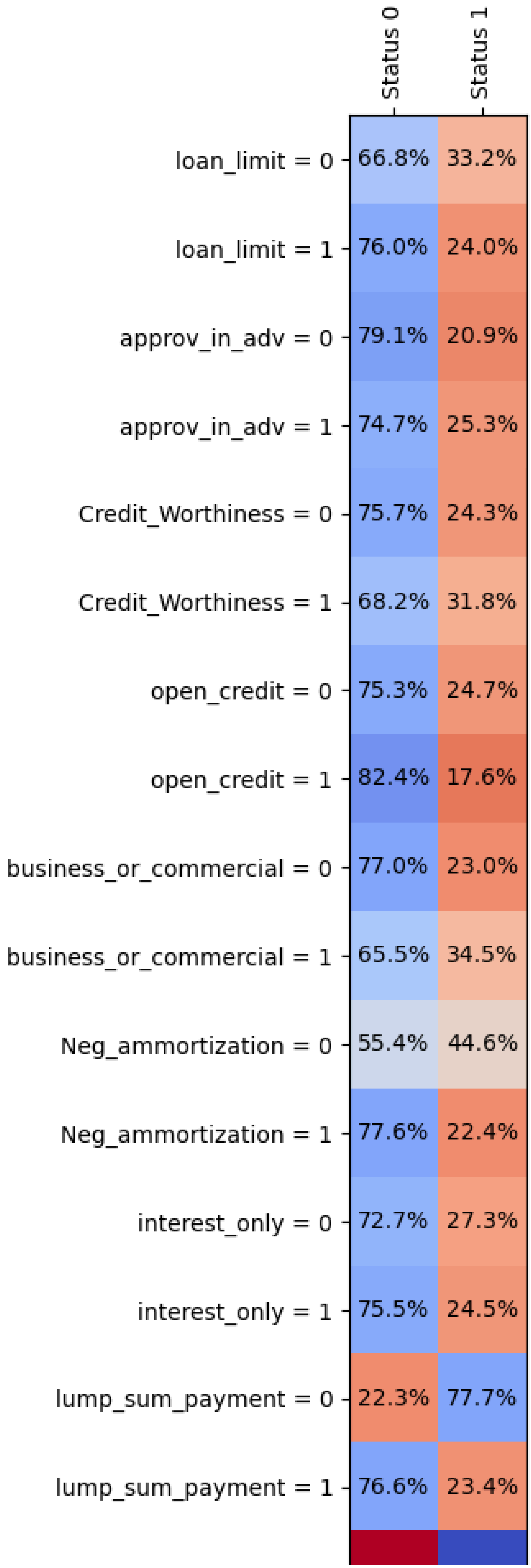
where:

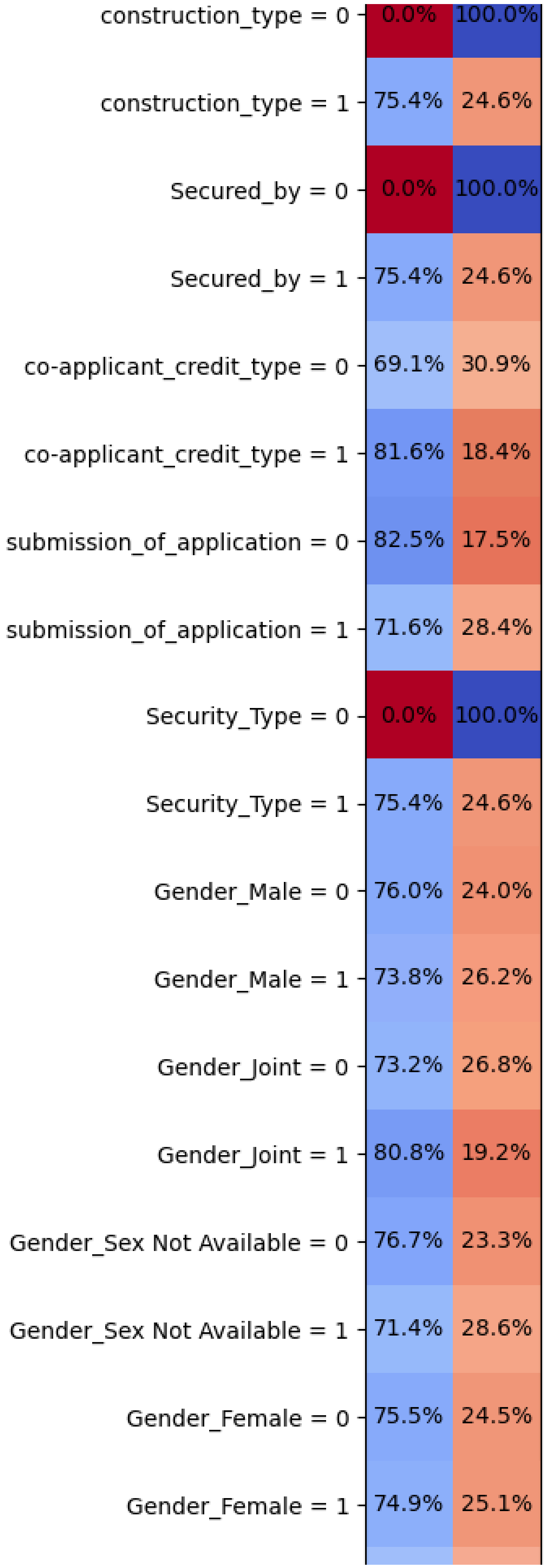
- cov is the covariance computed as: $cov(X,Y) \ = \ \mathbb{E}[(X-\mu_X)(Y-\mu_Y)]$
- σ_X is the standard deviation of X
- σ_Y is the sandard deviation of Y

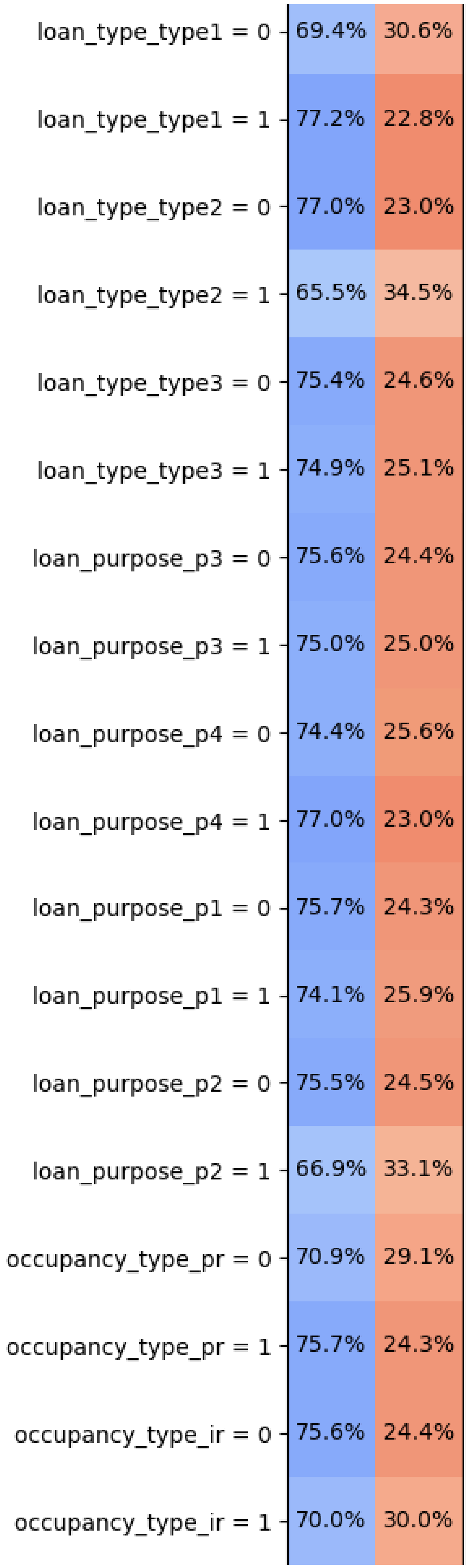
In the case of binary variables that assume only values in $\{0,1\}$, using Pearson's coefficient is a possible way to gain an overview of the correlation between variables [ResearchGate correlation of binary data](#).
Keep in mind that in literature there exist many other methods to achieve so, such as: point-biserial correlation coefficient, Spearman's correlation coefficient.

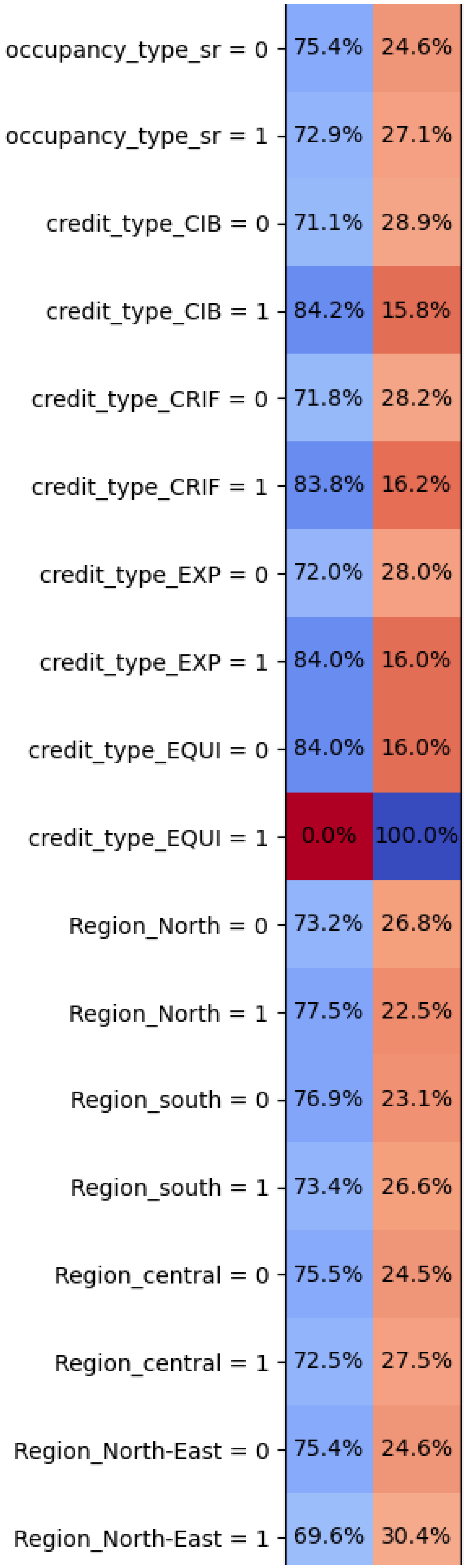
In [21]:

```
def compute_binary_feature_target_distribution(df, binary_features, target='Status'
    rows = []
    for feat in binary_features:
        for val in [0, 1]:
            #filter for feature value
            subset = df[(df[feat] == val) & (df[target].isin([0, 1]))]

            total = len(subset)
            count_0 = subset[subset[target] == 0].shape[0]
            count_1 = subset[subset[target] == 1].shape[0]

            pct_0 = (count_0 / total) * 100 if total > 0 else 0
            pct_1 = (count_1 / total) * 100 if total > 0 else 0

            rows.append({
                'feature': feat,
                'value': val,
                'Status 0 %': pct_0,
                'Status 1 %': pct_1
            })
    return pd.DataFrame(rows)


result_bin = compute_binary_feature_target_distribution(loan_data, categorical_feat

#extract data for heatmap
data_matrix = result_bin[['Status 0 %', 'Status 1 %']].to_numpy()
row_labels = [f"{f} = {v}" for f, v in zip(result_bin['feature'], result_bin['value
col_labels = ['Status 0', 'Status 1']

#plot
fig = plt.figure(figsize=(15, len(row_labels) * 0.7))
ax = fig.add_subplot(111)
cax = ax.matshow(data_matrix,
                 vmin=0,
                 vmax=100,
                 cmap='coolwarm_r')
# ticks
ax.set_xticks(np.arange(len(col_labels)))
ax.set_yticks(np.arange(len(row_labels)))
ax.set_xticklabels(col_labels, rotation=90)
ax.set_yticklabels(row_labels)

#annotate values
for i in range(data_matrix.shape[0]):
    for j in range(data_matrix.shape[1]):
        value = data_matrix[i, j]
        ax.text(j,
                i,
                f'{value:.1f}%',
                va='center',
                ha='center',
                color='black')

plt.show()
```


Status 0
Status 1
loan_limit = 0
66.8%
33.2%
loan_limit = 1
76.0%
24.0%
approv_in_adv = 0
79.1%
20.9%
approv_in_adv = 1
74.7%
25.3%
Credit_Worthiness = 0
75.7%
24.3%
Credit_Worthiness = 1
68.2%
31.8%
open_credit = 0
75.3%
24.7%
open_credit = 1
82.4%
17.6%
business_or_commercial = 0
77.0%
23.0%
business_or_commercial = 1
65.5%
34.5%
Neg_ammortization = 0
55.4%
44.6%
Neg_ammortization = 1
77.6%
22.4%
interest_only = 0
72.7%
27.3%
interest_only = 1
75.5%
24.5%
lump_sum_payment = 0
22.3%
77.7%
lump_sum_payment = 1
76.6%
23.4%


construction_type = 0
0.0%
100.0%
construction_type = 1
75.4%
24.6%
Secured_by = 0
0.0%
100.0%
Secured_by = 1
75.4%
24.6%
co-applicant_credit_type = 0
69.1%
30.9%
co-applicant_credit_type = 1
81.6%
18.4%
submission_of_application = 0
82.5%
17.5%
submission_of_application = 1
71.6%
28.4%
Security_Type = 0
0.0%
100.0%
Security_Type = 1
75.4%
24.6%
Gender_Male = 0
76.0%
24.0%
Gender_Male = 1
73.8%
26.2%
Gender_Joint = 0
73.2%
26.8%
Gender_Joint = 1
80.8%
19.2%
Gender_Sex Not Available = 0
76.7%
23.3%
Gender_Sex Not Available = 1
71.4%
28.6%
Gender_Female = 0
75.5%
24.5%
Gender_Female = 1
74.9%
25.1%


loan_type_type1 = 0
69.4%
30.6%
loan_type_type1 = 1
77.2%
22.8%
loan_type_type2 = 0
77.0%
23.0%
loan_type_type2 = 1
65.5%
34.5%
loan_type_type3 = 0
75.4%
24.6%
loan_type_type3 = 1
74.9%
25.1%
loan_purpose_p3 = 0
75.6%
24.4%
loan_purpose_p3 = 1
75.0%
25.0%
loan_purpose_p4 = 0
74.4%
25.6%
loan_purpose_p4 = 1
77.0%
23.0%
loan_purpose_p1 = 0
75.7%
24.3%
loan_purpose_p1 = 1
74.1%
25.9%
loan_purpose_p2 = 0
75.5%
24.5%
loan_purpose_p2 = 1
66.9%
33.1%
occupancy_type_pr = 0
70.9%
29.1%
occupancy_type_pr = 1
75.7%
24.3%
occupancy_type_ir = 0
75.6%
24.4%
occupancy_type_ir = 1
70.0%
30.0%



occupancy_type_sr = 0
75.4%
24.6%
occupancy_type_sr = 1
72.9%
27.1%
credit_type_CIB = 0
71.1%
28.9%
credit_type_CIB = 1
84.2%
15.8%
credit_type_CRIF = 0
71.8%
28.2%
credit_type_CRIF = 1
83.8%
16.2%
credit_type_EXP = 0
72.0%
28.0%
credit_type_EXP = 1
84.0%
16.0%
credit_type_EQUI = 0
84.0%
16.0%
credit_type_EQUI = 1
0.0%
100.0%
Region_North = 0
73.2%
26.8%
Region_North = 1
77.5%
22.5%
Region_south = 0
76.9%
23.1%
Region_south = 1
73.4%
26.6%
Region_central = 0
75.5%
24.5%
Region_central = 1
72.5%
27.5%
Region_North-East = 0
75.4%
24.6%
Region_North-East = 1
69.6%
30.4%

3. Data Preprocessing

So far we have handled the whole dataset manually in order to better manipulate it and visualize relevant characteristics.
In machine learning, it is way better to use **pipelines**, which are automated and organized series of steps that orchestrate the entire machine learning lifecycle, from data collection and preprocessing to model training, validation, and deployment.

In [22]:

```
#import of packages and libraries useful for fitting, transforming hence pipelines
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import FunctionTransformer
from sklearn.impute import SimpleImputer
import warnings #for ignoring the warnings while the models run
```

In [23]:

```
#while running some code blocks we encounter some useless warnings that in producti
#ignore the FutureWarning
warnings.filterwarnings("ignore", category=FutureWarning)
#ignore the UserWarning
#warnings.filterwarnings("ignore", category=UserWarning)
```

In [24]:

```
#features are now only numerical and ordinal, the number of them increased, as the
loan_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 148670 entries, 0 to 148669
Data columns (total 49 columns):
 #   Column                     Non-Null Count   Dtype  
---  ------                     --------------   -----  
 0   loan_limit                 145326 non-null  float64
 1   approv_in_adv              147762 non-null  float64
 2   Credit_Worthiness          148670 non-null  int64  
 3   open_credit                148670 non-null  int64  
 4   business_or_commercial     148670 non-null  int64  
 5   loan_amount                148670 non-null  int64  
 6   rate_of_interest           112231 non-null  float64
 7   Interest_rate_spread       112031 non-null  float64
 8   Upfront_charges            109028 non-null  float64
 9   term                       148629 non-null  float64
 10  Neg_ammortization          148549 non-null  float64
 11  interest_only              148670 non-null  int64  
 12  lump_sum_payment           148670 non-null  int64  
 13  property_value             133572 non-null  float64
 14  construction_type          148670 non-null  int64  
 15  Secured_by                 148670 non-null  int64  
 16  total_units                148670 non-null  float64
 17  income                     139520 non-null  float64
 18  Credit_Score               148670 non-null  int64  
 19  co-applicant_credit_type   148670 non-null  int64  
 20  age                        148470 non-null  float64
 21  submission_of_application  148470 non-null  float64
 22  LTV                        133572 non-null  float64
 23  Security_Type              148670 non-null  int64  
 24  Status                     148670 non-null  int64  
 25  dtir1                      124549 non-null  float64
 26  Gender_Female              148670 non-null  float64
 27  Gender_Joint               148670 non-null  float64
 28  Gender_Male                148670 non-null  float64
 29  Gender_Sex Not Available   148670 non-null  float64
 30  loan_type_type1            148670 non-null  float64
 31  loan_type_type2            148670 non-null  float64
 32  loan_type_type3            148670 non-null  float64
 33  loan_purpose_p1            148670 non-null  float64
 34  loan_purpose_p2            148670 non-null  float64
 35  loan_purpose_p3            148670 non-null  float64
 36  loan_purpose_p4            148670 non-null  float64
 37  loan_purpose_nan           148670 non-null  float64
 38  occupancy_type_ir          148670 non-null  float64
 39  occupancy_type_pr          148670 non-null  float64
 40  occupancy_type_sr          148670 non-null  float64
 41  credit_type_CIB            148670 non-null  float64
 42  credit_type_CRIF           148670 non-null  float64
 43  credit_type_EQUI           148670 non-null  float64
 44  credit_type_EXP            148670 non-null  float64
 45  Region_North               148670 non-null  float64
 46  Region_North-East          148670 non-null  float64
 47  Region_central             148670 non-null  float64
 48  Region_south               148670 non-null  float64
dtypes: float64(37), int64(12)
memory usage: 55.6 MB
```

Notice: the memory usage given directly by Pandas has increased by 17+ MB, due to the transformation into ordinal data.
In order to have a clearer idea of the actual memory used, as already mentioned in section 2.1. Dataset Description, we should use `sys.gestsizeof()` as below.

In [25]:
```
#the result is in bytes
sys.getsizeof(loan_data)
```

Out[25]: 58278784

The number shows that actually the memory used is way less with respect to the beginning, this is due to the fact that integer objects take less memory compared to string objects.

Since we want to embrace the *pipeline* approach, **we drop the previously manually manipulated dataset, and we take a new one, as it follows.**

In [26]:
```
try:
    loan_data = pd.read_csv(path + '\Loan_Default.csv')

except:
    loan_data = pd.read_csv('Loan_Default.csv') #if something goes wrong. use the c
```

In the following code block we apply column removal, based on the characteristics that emerged in sections: 2.3. and 2.4..
It is important to point out that the column removal has been applied on the whole dataset, because the criticalities on those features were so strong that allowed us to procede without compromising the effectiveness of the test set.
In order to better explain how the decisions have been taken we divide the data into numerical and categorical:

- **Numerical**:
  - `ID` , `year` : have been removed since they did not bring any relevant information for predictive purposes.
  - `rate_of_interest` , `Interest_rate_spread` , `Upfront_charges` : have been removed, because they were strongly compromised, they have the highest missing values and furthermore whenever they are not missing, the target is nearly always: `Status=0`
- **Categorical**:
  - `business_or_commercial` : has been removed because from the correlation matrix emerges that is strongly correlated to `loan_type_2`
  - `Secured_by` , `Security_Type` : have been removed because from the correlation matrix emerged that they were strongly correlated to each other and to `construction_type` , furthermore whenever `Secured_by=land` or `Security_Type=indirect` the target always assumes: `Status=1`

- When the feature `credit_type` assumes the value `EQUI`, the target is always `Status=1`, in this situation it is spuriously predictive. It is a case of data leakage), by removing it, we prevent the model from learning this allegedly wrong pattern. `credit_type` is later transformed with one-hot encoding, because it can assume four different values, so we decided to set it to `NaN` whenever it assumes `EQUI`. Later, when the models call the *fit* of the *pipeline* on only the training set, the `NaN` will be replaced with the most frequent category.

It is important to point out that the threshold of correlation by which we decided to remove columns has been set to \$0.90\$, since decision trees and even better ensemble learning algorithms are quite robust when they face multicollinearity, as opposed to other simpler models such as logistic regression.

In [27]:
```
#removing the non-relevant features
loan_data = loan_data.drop([#numerical
                        'ID',
                        'year',
                        'rate_of_interest',
                        'Interest_rate_spread',
                        'Upfront_charges',
                         #categorical
                        'Security_Type',
                        'Secured_by',
                        'business_or_commercial',
                           ],
                           axis=1
                          )

#setting to NaN the non-relevant columns
loan_data.loc[loan_data['credit_type'] == 'EQUI', 'credit_type'] = np.nan
```

In [28]:
```
#map for the ordinal mapping of age groups
age_mapping = {'<25': 1,
               '25-34': 2,
               '35-44': 3,
               '45-54': 4,
               '55-64': 5,
               '65-74': 6,
               '>74': 7
               }
#map for the ordinal mapping of the total_units
total_units_mapping = {'1U': 1,
                   '2U': 2,
                   '3U': 3,
                   '4U': 4
               }

#list of the names of the ordinal features
ordinal_features = ['age',
               'total_units',
              ]
#list of the names of the categorical features where we will apply OneHotEncoding
```

```
onehot_features = ['loan_limit',
                   'approv_in_adv',
                   'Credit_Worthiness',
                   'open_credit',
                   'Neg_ammortization',
                   'interest_only',
                   'lump_sum_payment',
                   'construction_type',
                   'co-applicant_credit_type',
                   'submission_of_application',
                   'Gender',
                   'loan_type',
                   'loan_purpose',
                   'occupancy_type',
                   'credit_type',
                   'Region',
                  ]
#list of the numerical features
numerical_features = ['loan_amount',
                      'term',
                      'property_value',
                      'income',
                      'Credit_Score',
                      'LTV',
                      'dtir1'
                     ]
```

In [29]:

```
# Categorical one-hot
cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore')) #we ignore the unknown cathe
])

# Numerical
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median'))
])

# Ordinal
#age ordinal pipeline
age_pipeline = Pipeline([
    ('mapper', FunctionTransformer(
        lambda x: x.iloc[:, 0].map(age_mapping).to_frame(), #anonymous function, th
        validate=False)),                                    #the mapping we have pr
    ('imputer', SimpleImputer(strategy='most_frequent'))
])
#total_units ordinal pipeline
total_units_pipeline = Pipeline([
    ('mapper', FunctionTransformer(
        lambda x: x.iloc[:, 0].map(total_units_mapping).to_frame(),
        validate=False)), #since we want to work with DataFrame we must set validat
    ('imputer', SimpleImputer(strategy='most_frequent'))
])

#Full pipeline, merging all type of data
preprocessor = ColumnTransformer([
```

```
    ('cat_onehot', cat_pipeline, onehot_features),
    ('num', num_pipeline, numerical_features),
    ('age_ord', age_pipeline, ['age']), #since ColumnTransformer apply transformati
    ('units_ord', total_units_pipeline, ['total_units']),

])
```

In [30]:

```
#memory usage in bytes of the processed dataset with the pipeline
sys.getsizeof(preprocessor.fit_transform(loan_data))
```

Out[30]:

```
59468120
```

Notice: now that we have applied the `preprocessor` (that is a pipeline for column transformation) to the original dataset, we see that the memory used is again way less with respect to the original one, buth slightly more compared to the manually transformed dataset obtained at the end of section 2. (that contained 49 columns), even though now we have removed nine features.
This is simply due to the fact that with the `preprocessor` the one-hot encoding is fully applied to all categorical data (except: `age`, `total_units`), resulting with 50 columns.

4. Training the Models

In [31]:

```
#import of all libraries and packages
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from xgboost import XGBClassifier #XGBoostClassifier
from sklearn.ensemble import AdaBoostClassifier #AdaBoost
from sklearn.tree import DecisionTreeClassifier #for AdaBoost
from sklearn.ensemble import GradientBoostingClassifier #GradientBoost
#instead of Joblib, we use Cloudpickle, since it is able to save also the lambda fu
import cloudpickle #for saving the models
#for getting all the metrics and displaying them
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, average_preci
import time #for counting the amount of time it takes for the algorithm to run
```

In [32]:

```
#for converting seconds into readable format, for the runtime
def convert_time(seconds):
    seconds = seconds % (24 * 3600)
    hour = seconds // 3600
    seconds %= 3600
    minutes = seconds // 60
    seconds %= 60

    return "%d:%02d:%02d" % (hour, minutes, seconds)
```

In [33]:

```
#Separate features and target
X = loan_data.drop(columns=['Status'])
y = loan_data['Status']

#split the dataset into train and test sets, with a proportion of 90-10%, since we
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.10,
```

```
                                                    random_state=42,
                                                    stratify=y, #in order to keep t
                                                   )


#stratified K-Fold for balanced class distribution over the target
strat_kfold = StratifiedKFold(n_splits=5, #number of folds
                              shuffle=True, #before creating the fold, it shuffles
                              random_state=42, #footnote following block
                             )
```

Notice: our dataset is a bit unbalanced in terms of target values, since the samples with `Status=0` are way more than the ones with `Status=1`, this can lead to slighlty biased models. In order to overcome this issue we tweak some models' paramaters accordingly.

This practice is especially useful for improving the `recall` of our models.

$$ Recall = \frac{\text{True Positives}}{\text{True Positives}\ +\ \text{False Negatives}} $$

As opposed to:

$$ Precision = \frac{\text{True Positives}}{\text{True Positives}\ +\ \text{False Positives}} $$

Since credit institutes tend to be risk adverse in front of potential loan clients, they ***prefer recall over precision***, and this is the way we approach our three following models.

`random_state = 42` is an inside joke of machine learning base on the famous book: *The Hitchhiker's Guide to the Galaxy, Douglas Adams*, see

4.1. eXtreme Gradient Boosting

In [34]:
```
#counting the number of positive and negative samples for balancing the model
num_negatives = loan_data['Status'].value_counts()[0]
num_positives = loan_data['Status'].value_counts()[1]
```

In [35]:
```
"""
Notice, if you do not have enough time, you can skip directly to the following cell
object in your folder, since it will open the already trained model.
"""

#setting the XGBoost classifier
xgb_model = XGBClassifier(eval_metric='logloss', #cost funtion
                          random_state=42, #to control the internal random componen
                          scale_pos_weight = num_negatives / num_positives #it give
                         )                                                 #since t
                                                                           #unbalan
#adding to the pipeline the algorithm
full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', xgb_model)
```

```
])

#parameter grid for GridSearch
param_grid = {
    'classifier__n_estimators': [50, 100, 200],
    'classifier__max_depth': [5, 6, 7],
    'classifier__learning_rate': [0.1, 0.3, 0.5],
    'classifier__subsample': [0.8, 1],  #fraction of samples used for fitting each
}

# Grid search setup
grid_search_XGBoost = GridSearchCV(full_pipeline,
                           param_grid,
                           cv=strat_kfold,
                           n_jobs=-1, #it uses all the CPU cores availbale in paral
                           verbose=2, #it shows all details for each combination
                           scoring = 'recall', #since we want prioritize recall
                           #scoring = 'f1',    #even this one leads very good resul
                                 )

start_time = time.time() #for counting the time
# Fit GridSearchCV on training data
grid_search_XGBoost.fit(X_train,
                y_train
               )
end_time = time.time() #end time of execution
elapsed_time = convert_time(end_time - start_time)
print(f"XGBoosting training + grid search took {elapsed_time}")

# Best params and score
print("Best parameters found:", grid_search_XGBoost.best_params_)
print("Best cross-validation Recall:", grid_search_XGBoost.best_score_)

# Optional: Evaluate on the test set
test_score = grid_search_XGBoost.score(X_test, y_test)
print(f"Test set Recall: {test_score:.4f}") #we print the accuracy with the four de
```

```
Fitting 5 folds for each of 54 candidates, totalling 270 fits
XGBoosting training + grid search took 0:16:58
Best parameters found: {'classifier__learning_rate': 0.5, 'classifier__max_depth':
5, 'classifier__n_estimators': 50, 'classifier__subsample': 0.8}
Best cross-validation Recall: 0.7280060652009098
Test set Recall: 0.7481
```

In [36]:

```
#try to see whether the above cell has been run
try:
    grid_search_XGBoost
    recovered = False #boolean variable to understand wheter the model has been rec

#otherwise open the serialized model
except:
    with open("XGBoost_loan.pkl", mode="rb") as f: #rb stands for reading binary
        grid_search_XGBoost = cloudpickle.load(f)
    recovered = True
```

In [37]:

```
pd.DataFrame(grid_search_XGBoost.cv_results_)
```

Out[37]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__learning_
0	9.959908	0.111911	0.802918	0.367341	
1	12.227372	3.899827	1.755331	0.328310	
2	23.036407	2.366370	1.297397	0.365357	
3	14.486458	1.880278	0.800972	0.278468	
4	17.239578	0.220814	0.870704	0.023898	
5	13.535185	0.175856	0.833076	0.028101	
6	7.789968	0.249640	0.629102	0.022814	
7	6.889371	0.064463	0.632961	0.054496	
8	11.090194	0.225003	0.715223	0.050741	
9	9.592282	0.045793	0.728181	0.063756	
10	18.012373	0.280872	0.902797	0.023970	
11	14.426612	0.122053	0.922483	0.030842	
12	8.038273	0.083363	0.695766	0.046859	
13	8.249799	0.644209	0.918591	0.184402	
14	20.633695	0.349632	1.220312	0.245398	
15	19.012687	0.470595	1.817459	0.784448	
16	27.048638	2.356832	1.199594	0.087897	
17	16.867118	1.443681	1.051921	0.073208	
18	7.360573	0.173280	0.609933	0.043352	

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__learning_
19	6.379111	0.076008	0.625277	0.061071	
20	10.775345	0.145443	0.721807	0.046877	
21	8.764243	0.168277	0.764832	0.094291	
22	16.549371	0.394369	0.845649	0.029795	
23	13.160515	0.536135	0.845918	0.028440	
24	7.339692	0.071558	0.615039	0.022919	
25	6.421416	0.075103	0.631852	0.025604	
26	10.895021	0.267455	0.732253	0.027924	
27	9.400118	0.199299	0.739972	0.036621	
28	19.909610	1.870109	0.994584	0.084985	
29	16.780405	0.524896	1.108583	0.126493	
30	8.864072	0.614887	0.712018	0.032819	
31	7.547914	0.119703	0.742236	0.049442	
32	16.395838	3.965111	1.662178	0.612789	
33	15.784762	3.779051	0.977600	0.050874	
34	23.934121	0.578148	1.415445	0.218974	
35	20.852833	1.539357	1.450688	0.298924	
36	9.749824	0.915292	0.951257	0.210750	
37	7.901964	0.537534	0.726036	0.071961	

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__learning_
38	12.846456	0.619731	0.952467	0.215300	
39	11.274114	1.802516	0.901988	0.160800	
40	27.085224	1.745152	1.155628	0.145150	
41	15.034133	1.750489	0.927211	0.134312	
42	7.424155	0.068395	0.653203	0.059335	
43	6.470957	0.096106	0.635905	0.037022	
44	10.870731	0.155201	0.740229	0.048882	
45	8.945145	0.131690	0.698178	0.015774	
46	18.597185	0.646536	0.917283	0.048571	
47	17.403107	2.578652	2.432832	1.788268	
48	8.154314	0.118954	0.729547	0.076788	
49	6.850456	0.116967	0.663318	0.039920	
50	11.778309	0.121604	0.783455	0.049557	
51	9.747708	0.058044	0.772599	0.062101	
52	20.359479	0.383227	1.114856	0.112094	
53	15.291070	1.323871	0.957885	0.215121	

In [38]:
```
#we encapsulate just the best model
XGBoost_whole_model = grid_search_XGBoost.best_estimator_ #here we save the whole b
XGBoost_classifier = XGBoost_whole_model.named_steps['classifier'] #here we save ju
```

In [39]:
```
#recover just the preprocessor from the whole model
preprocessor = XGBoost_whole_model.named_steps['preprocessor']
feature_names = []
```

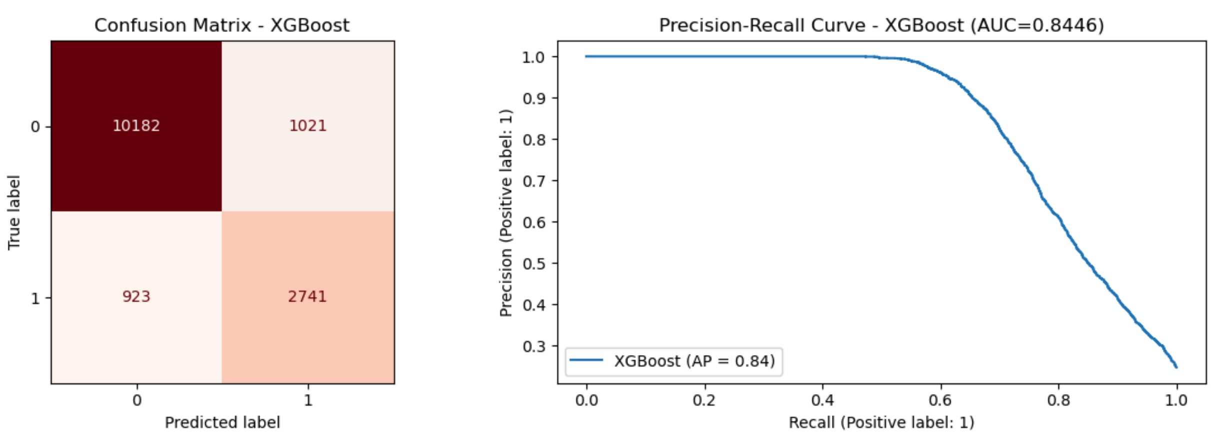
```
for name, transformer, cols in preprocessor.transformers_:
    #if the transformer is a drop operator
    if transformer == 'drop':
        continue
    #if the transformer is actually a pipeline object, so that has inside other tra
    if isinstance(transformer, Pipeline):
        #get the last step of the pipeline, because the last step is the one that f
        last_step = transformer.steps[-1][1]
        if hasattr(last_step, 'get_feature_names_out'):
            names = last_step.get_feature_names_out(cols)
        else:
            names = cols
    #if the transformer is directly a transformer
    else:
        if hasattr(transformer, 'get_feature_names_out'):
            names = transformer.get_feature_names_out(cols)
        else:
            names = cols

    feature_names.extend(names)

feat_imp = pd.DataFrame({
                        'feature': feature_names,
                        'importance': XGBoost_classifier.feature_importances_
                        }).sort_values(
                                        by="importance",
                                        ascending=False
                                        )

print(feat_imp)
```

```
                                 feature  importance
12                   lump_sum_payment_lpsm    0.136860
43                          property_value    0.125512
34                         credit_type_CIB    0.121303
8                  Neg_ammortization_neg_amm    0.079789
46                                     LTV    0.054352
18   submission_of_application_not_inst    0.053599
4                      Credit_Worthiness_l1    0.034651
26                         loan_type_type3    0.031678
47                                   dtir1    0.029537
24                         loan_type_type1    0.027752
25                         loan_type_type2    0.027425
2                      approv_in_adv_nopre    0.021031
0                            loan_limit_cf    0.020551
10                 interest_only_int_only    0.017750
27                         loan_purpose_p1    0.016836
44                                  income    0.016434
49                             total_units    0.015406
32                       occupancy_type_pr    0.015095
21                            Gender_Joint    0.013179
28                         loan_purpose_p2    0.013005
29                         loan_purpose_p3    0.012184
37                            Region_North    0.011764
14                    construction_type_mh    0.010875
31                       occupancy_type_ir    0.010173
41                             loan_amount    0.010054
30                         loan_purpose_p4    0.009058
16             co-applicant_credit_type_CIB    0.008699
33                       occupancy_type_sr    0.005896
42                                    term    0.005726
48                                     age    0.005487
39                          Region_central    0.005393
35                        credit_type_CRIF    0.004872
23                  Gender_Sex Not Available    0.004803
6                         open_credit_nopc    0.004096
22                             Gender_Male    0.004086
45                            Credit_Score    0.003525
40                            Region_south    0.003518
20                           Gender_Female    0.003083
36                         credit_type_EXP    0.002692
38                       Region_North-East    0.002272
15                    construction_type_sb    0.000000
9                  Neg_ammortization_not_neg    0.000000
3                       approv_in_adv_pre    0.000000
5                      Credit_Worthiness_l2    0.000000
17             co-applicant_credit_type_EXP    0.000000
7                          open_credit_opc    0.000000
19    submission_of_application_to_inst    0.000000
1                           loan_limit_ncf    0.000000
11                   interest_only_not_int    0.000000
13                 lump_sum_payment_not_lpsm    0.000000
```

In [40]:

```
# Predict on the test set
y_pred = grid_search_XGBoost.predict(X_test)

# Compute confusion matrix
```

```python
cm = confusion_matrix(y_test, y_pred)

# Get predicted probabilities for the positive class
y_scores = grid_search_XGBoost.predict_proba(X_test)[:, 1]

# Compute average precision (AUC-PR)
auc_pr = average_precision_score(y_test, y_scores)

# Create a figure
fig, axes = plt.subplots(1, # one row
                         2, # two columns
                         figsize=(12, 4)
                        )

# --- Confusion Matrix ---
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=grid_search_XGBoost.classes_
                             )
disp.plot(cmap="Reds",
          values_format='d',
          colorbar=False, #don't show the legend colormap
          ax=axes[0])
axes[0].set_title("Confusion Matrix - XGBoost")

# --- Precision-Recall Curve ---
PrecisionRecallDisplay.from_predictions(y_test,
                                        y_scores,
                                        name="XGBoost",
                                        ax=axes[1]
                                       )
axes[1].set_title(f"Precision-Recall Curve - XGBoost (AUC={auc_pr:.4f})")

plt.tight_layout()
plt.show()
```

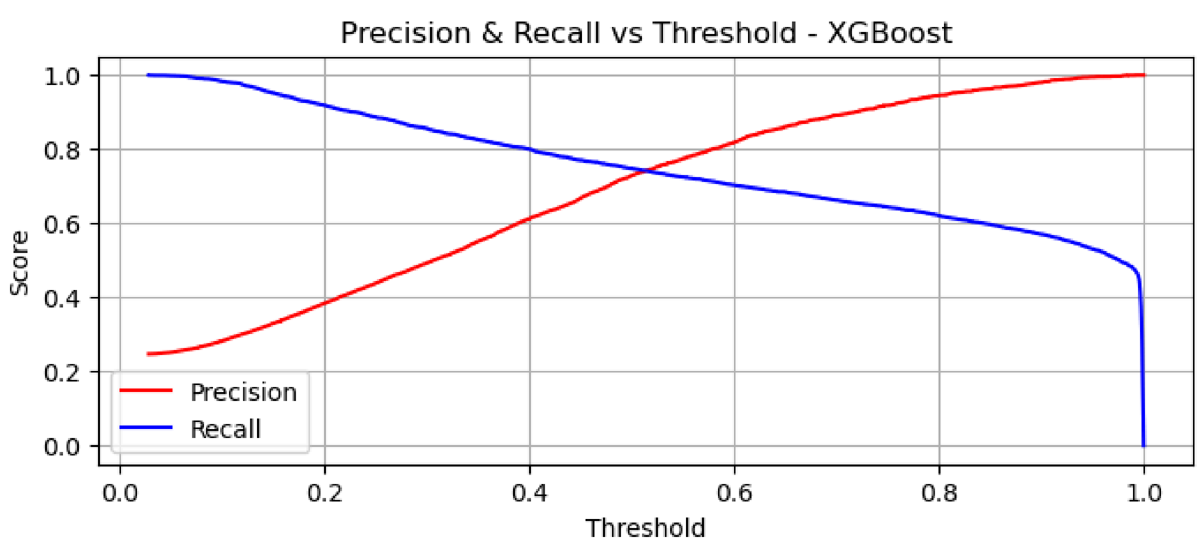


Notice: we kept the same balance of the original dataset based on target values for our test set, thanks to the parameter: `stratify=y` during the split of the dataset in section 4.. Thus, the preportion of: $ \frac{\text{negatives}}{\text{positives}}=3.05 $, is kept on both training and testing sets.

In [41]:
```
# Predict on test set
y_pred = grid_search_XGBoost.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)
# Precision (by default, for positive class in binary classification)
prec = precision_score(y_test, y_pred)
# Recall
rec = recall_score(y_test, y_pred)
# F1-score
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1-score: {f1:.4f}")
```

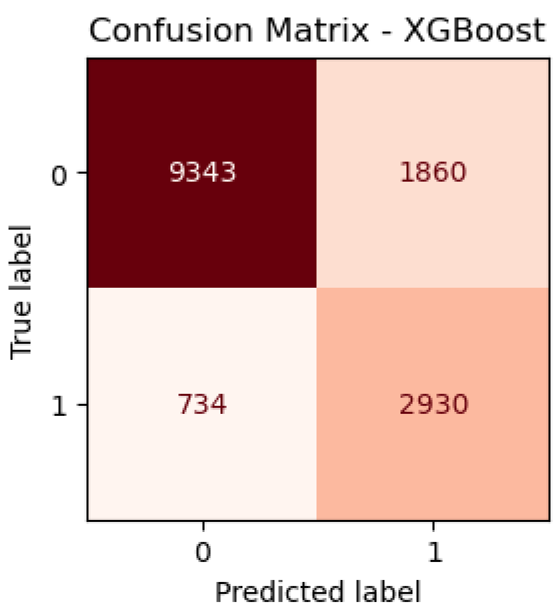
```
Accuracy: 0.8692
Precision: 0.7286
Recall: 0.7481
F1-score: 0.7382
```

In [42]:
```
# Get predicted probabilities for the positive class
y_scores = grid_search_XGBoost.predict_proba(X_test)[:, 1]

# Compute precision, recall, thresholds
precision, recall, thresholds = precision_recall_curve(y_test, y_scores)

# Plot Precision and Recall vs Threshold
plt.figure(figsize=(8, 3))
plt.plot(thresholds, precision[:-1], label='Precision', color='red')
plt.plot(thresholds, recall[:-1], label='Recall', color='blue')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision & Recall vs Threshold - XGBoost')
plt.legend()
plt.grid(True)
plt.show()
```



In [43]:
```
y_scores = grid_search_XGBoost.predict_proba(X_test)[:, 1]
threshold = 0.4  # lower than 0.5, where it is centered
y_pred_adjusted = (y_scores >= threshold).astype(int)
```

In [44]:
```
# Compute confusion matrix
cm = confusion_matrix(y_test,
                      y_pred_adjusted
                     )

# Display confusion matrix as a heatmap
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=grid_search_XGBoost.classes_
                             )
plt.figure(figsize=(3, 3)) #create a specific figure object in order to better mani

disp.plot(cmap="Reds",
          values_format='d', #show numbers as integers
          colorbar = False, #colorbar as legend disactivated
          ax=plt.gca() #plot inn the figure created
         )

plt.title("Confusion Matrix - XGBoost")
plt.show()
```



In [45]:
```
# Accuracy
acc = accuracy_score(y_test, y_pred_adjusted)
# Precision (by default, for positive class in binary classification)
prec = precision_score(y_test, y_pred_adjusted)
# Recall
rec = recall_score(y_test, y_pred_adjusted)
# F1-score
f1 = f1_score(y_test, y_pred_adjusted)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
```

```
print(f"Recall: {rec:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
Accuracy: 0.8255
Precision: 0.6117
Recall: 0.7997
F1-score: 0.6932
```

The following code block performs *serialization*, it is a very helpful practice that allows to save a model once has been trained, so it saves the training time every time you open the notebook by simply opening directly the saved model.
There are many libraries that apply serialization and probably one of the best-known is **`pickle`**, even though it is not able to save **`lambda`** functions when present.
Since this practice can also save the full pipeline, and in our case there are **`lambda`** functions, we ended up using **`cloudpickle`**, that also saves the anonymous functions.
The storing format is binary.

In [46]:
```
# --- Saving ---
if recovered is False: #if the model currently used has not been recovered from ser
    with open("XGBoost_loan.pkl", mode="wb") as f: #wb stands for Writing Binary
        cloudpickle.dump(grid_search_XGBoost, f)

# --- Loading ---
"""
with open("XGBoost_loan.pkl", mode="rb") as f: #rb stands for reading binary
    XGBoost_recovered = cloudpickle.load(f)
"""
```

Out[46]:
```
'\nwith open("XGBoost_loan.pkl", mode="rb") as f: #rb stands for reading binary\n
XGBoost_recovered = cloudpickle.load(f)\n'
```

4.2. Adaptive Boosting

In [47]:
```
"""
Notice, if you do not have enough time, you can skip directly to the following cell
object in your folder, since it will open the already trained model.
"""

#Setting AdaBoost classifier (with Decision Tree as base estimator)
ada_model = AdaBoostClassifier(
                                base_estimator=DecisionTreeClassifier(class_weight=
                                                                     ),
                                random_state=42
                            )

#adding to the pipeline the algorithm
full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', ada_model)
])

# Parameter grid for GridSearch
param_grid = {
```

```
    'classifier__base_estimator__max_depth': [3, 4, 5],       #Maximum depth of dec
    'classifier__n_estimators': [150, 200, 300],            #Number of boosting sta
    'classifier__learning_rate': [0.01, 0.3, 0.1],    #Shrinkage (learning) rate
    'classifier__algorithm': [#'SAMME', # Boosting algorithms, for our case SAMME.R
                              'SAMME.R']
}

# Grid search setup
grid_search_AdaBoost = GridSearchCV(
                            full_pipeline,
                            param_grid,
                            cv=strat_kfold,
                            n_jobs=-1,
                            verbose=2,
                            scoring='recall',
                        )

start_time = time.time() #for counting the time
# Fit GridSearchCV on training data
grid_search_AdaBoost.fit(X_train,
                y_train
                )
end_time = time.time() #end time of execution
elapsed_time = convert_time(end_time - start_time)
print(f"AdaBoost training + grid search took {elapsed_time}")

# Best params and score
print("Best parameters found:", grid_search_AdaBoost.best_params_)
print("Best cross-validation Recall:", grid_search_AdaBoost.best_score_)

# Evaluate on the test set
test_score = grid_search_AdaBoost.score(X_test, y_test)
print(f"Test set Recall: {test_score:.4f}")
```

```
Fitting 5 folds for each of 27 candidates, totalling 135 fits
AdaBoost training + grid search took 4:45:10
Best parameters found: {'classifier__algorithm': 'SAMME.R', 'classifier__base_estima
tor__max_depth': 3, 'classifier__learning_rate': 0.3, 'classifier__n_estimators': 30
0}
Best cross-validation Recall: 0.7224867323730099
Test set Recall: 0.7424
```

In [48]:
```
#try to see whether the above cell has been run
try:
    grid_search_AdaBoost
    recovered = False #boolean variable to understand wheter the model has been rec

#otherwise open the serialized model
except:
    with open("AdaBoost_loan.pkl", mode="rb") as f: #rb stands for reading binary
        grid_search_AdaBoost = cloudpickle.load(f)
    recovered = True
```

In [49]:
```
pd.DataFrame(grid_search_AdaBoost.cv_results_)
```

Out[49]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__algorithr
0	245.091279	1.433680	4.111456	0.086499	SAMME.
1	317.415425	3.034288	5.322965	0.235682	SAMME.
2	472.035136	1.909110	7.570289	0.225191	SAMME.
3	238.743706	3.625973	4.291338	0.575546	SAMME.
4	364.822918	22.906856	6.342026	1.683179	SAMME.
5	466.482349	30.795482	6.480300	1.756797	SAMME.
6	244.422789	1.004910	4.046438	0.075939	SAMME.
7	321.772150	0.887950	5.266049	0.118813	SAMME.
8	515.447549	12.489158	10.032569	2.061255	SAMME.
9	332.836284	18.517381	3.979810	0.052671	SAMME.
10	410.594428	2.939652	5.124905	0.099056	SAMME.
11	653.750121	23.469345	7.898930	0.200846	SAMME.
12	307.835304	2.912320	3.975472	0.033814	SAMME.
13	446.468203	13.472946	5.554764	0.374650	SAMME.
14	634.295039	12.053299	7.712355	0.069699	SAMME.
15	322.399766	16.085456	4.129928	0.146897	SAMME.
16	438.795249	14.444886	6.547107	1.064233	SAMME.
17	638.956153	12.908956	8.337363	1.325155	SAMME.
18	406.464667	18.696185	4.304856	0.354997	SAMME.

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__algorithr
19	627.692060	33.060848	8.225628	0.734167	SAMME.
20	1103.511735	115.028874	13.411554	1.452328	SAMME.
21	654.125297	30.348780	7.687926	1.300510	SAMME.
22	624.216390	59.971174	5.803365	0.545394	SAMME.
23	720.234004	18.568695	7.333406	0.262206	SAMME.
24	345.514080	1.678799	3.768399	0.040220	SAMME.
25	463.378268	1.164982	4.973119	0.184906	SAMME.
26	617.579434	110.591147	6.869373	1.778494	SAMME.

In [50]:

```
#we encapsulate just the best model
AdaBoost_whole_model = grid_search_AdaBoost.best_estimator_ #here we save the whole
AdaBoost_classifier = AdaBoost_whole_model.named_steps['classifier'] #here we save
```

In [51]:

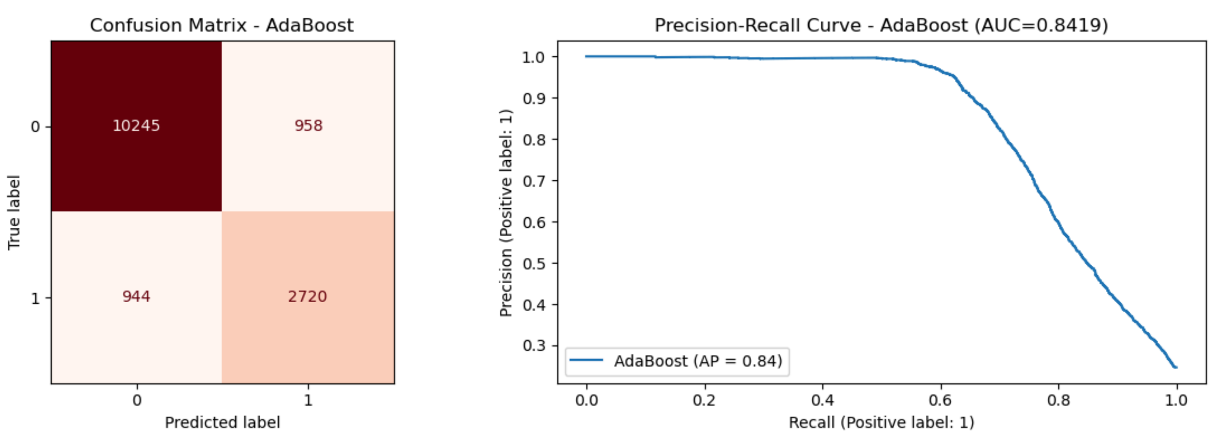
```
#recover just the preprocessor from the whole model
preprocessor = AdaBoost_whole_model.named_steps['preprocessor']
feature_names = []

for name, transformer, cols in preprocessor.transformers_:
    #if the transformer is a drop operator
    if transformer == 'drop':
        continue
    #if the transformer is actually a pipeline object, so that has inside other tra
    if isinstance(transformer, Pipeline):
        #get the last step of the pipeline, because the last step is the one that f
        last_step = transformer.steps[-1][1]
        if hasattr(last_step, 'get_feature_names_out'):
            names = last_step.get_feature_names_out(cols)
        else:
            names = cols
    #if the transformer is directly a transformer
    else:
        if hasattr(transformer, 'get_feature_names_out'):
            names = transformer.get_feature_names_out(cols)
        else:
            names = cols

    feature_names.extend(names)

feat_imp = pd.DataFrame({
```

```
                                'feature': feature_names,
                                'importance': AdaBoost_classifier.feature_importances_
                                }).sort_values(
                                                    by="importance",
                                                    ascending=False
                                                    )

print(feat_imp)
```

```
                               feature  importance
46                                 LTV    0.205134
44                              income    0.131380
43                      property_value    0.098443
47                               dtir1    0.090626
41                         loan_amount    0.078529
45                        Credit_Score    0.065378
29                     loan_purpose_p3    0.024724
48                                 age    0.024074
31                   occupancy_type_ir    0.022039
42                                term    0.021247
27                     loan_purpose_p1    0.018398
24                     loan_type_type1    0.017426
30                     loan_purpose_p4    0.017361
26                     loan_type_type3    0.011771
2                  approv_in_adv_nopre    0.010662
22                         Gender_Male    0.009872
21                        Gender_Joint    0.008940
9              Neg_ammortization_not_neg    0.008019
49                         total_units    0.008008
25                     loan_type_type2    0.007877
19    submission_of_application_to_inst    0.007611
8              Neg_ammortization_neg_amm    0.007359
17         co-applicant_credit_type_EXP    0.007241
4                  Credit_Worthiness_l1    0.007118
18   submission_of_application_not_inst    0.006882
3                    approv_in_adv_pre    0.006844
1                       loan_limit_ncf    0.005524
37                        Region_North    0.005429
13             lump_sum_payment_not_lpsm    0.005085
36                     credit_type_EXP    0.005038
5                  Credit_Worthiness_l2    0.004803
39                      Region_central    0.004575
28                     loan_purpose_p2    0.004568
16         co-applicant_credit_type_CIB    0.004443
10              interest_only_int_only    0.004397
34                     credit_type_CIB    0.004236
35                    credit_type_CRIF    0.004229
32                   occupancy_type_pr    0.004019
12                lump_sum_payment_lpsm    0.003556
33                   occupancy_type_sr    0.003234
20                       Gender_Female    0.003051
11               interest_only_not_int    0.002444
0                        loan_limit_cf    0.002260
23             Gender_Sex Not Available    0.001560
6                     open_credit_nopc    0.001486
7                      open_credit_opc    0.001085
40                        Region_south    0.001079
15                 construction_type_sb    0.000562
38                   Region_North-East    0.000374
14                 construction_type_mh    0.000000
```

In [52]:
```
# Predict on the test set
y_pred = grid_search_AdaBoost.predict(X_test)

# Compute confusion matrix
```

```python
cm = confusion_matrix(y_test, y_pred)

# Get predicted probabilities for the positive class
y_scores = grid_search_AdaBoost.predict_proba(X_test)[:, 1]

# Compute average precision (AUC-PR)
auc_pr = average_precision_score(y_test, y_scores)

# Create a figure
fig, axes = plt.subplots(1, # one row
                         2, # two columns
                         figsize=(12, 4)
                        )

# --- Confusion Matrix ---
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=grid_search_AdaBoost.classes_
                             )
disp.plot(cmap="Reds",
          values_format='d',
          colorbar=False, #don't show the legend colormap
          ax=axes[0])
axes[0].set_title("Confusion Matrix - AdaBoost")

# --- Precision-Recall Curve ---
PrecisionRecallDisplay.from_predictions(y_test,
                                        y_scores,
                                        name="AdaBoost",
                                        ax=axes[1]
                                       )
axes[1].set_title(f"Precision-Recall Curve - AdaBoost (AUC={auc_pr:.4f})")

plt.tight_layout()
plt.show()
```



In [53]:

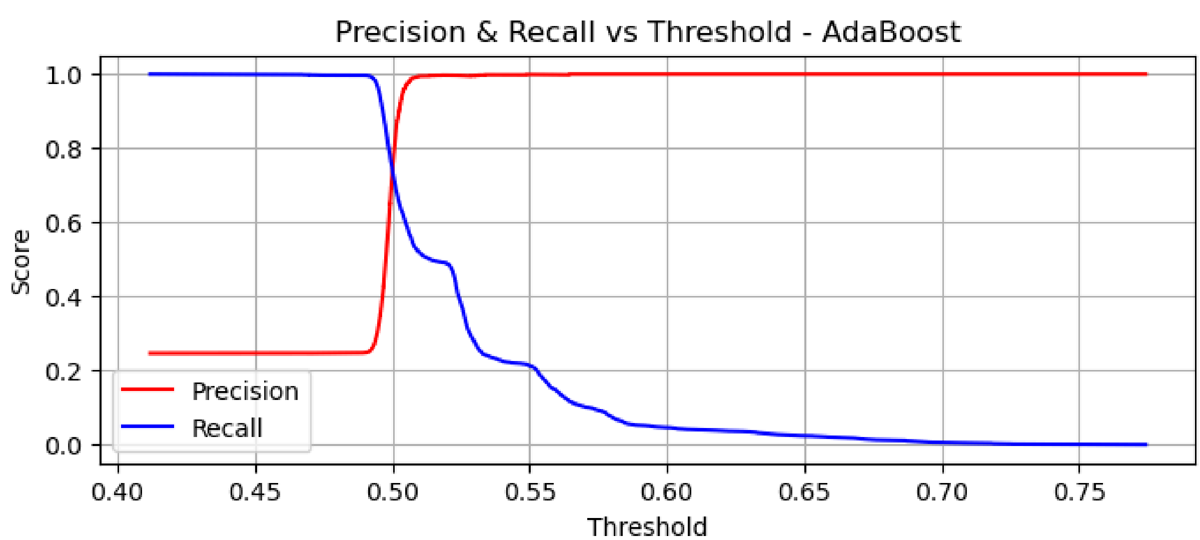
```python
# Predict on test set
y_pred = grid_search_AdaBoost.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)
# Precision (by default, for positive class in binary classification)
prec = precision_score(y_test, y_pred)
```

```python
# Recall
rec = recall_score(y_test, y_pred)
# F1-score
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
Accuracy: 0.8721
Precision: 0.7395
Recall: 0.7424
F1-score: 0.7409
```

In [54]:

```python
# Get predicted probabilities for the positive class
y_scores = grid_search_AdaBoost.predict_proba(X_test)[:, 1]

# Compute precision, recall, thresholds
precision, recall, thresholds = precision_recall_curve(y_test, y_scores)

# Plot Precision and Recall vs Threshold
plt.figure(figsize=(8, 3))
plt.plot(thresholds, precision[:-1], label='Precision', color='red')
plt.plot(thresholds, recall[:-1], label='Recall', color='blue')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision & Recall vs Threshold - AdaBoost')
plt.legend()
plt.grid(True)
plt.show()
```



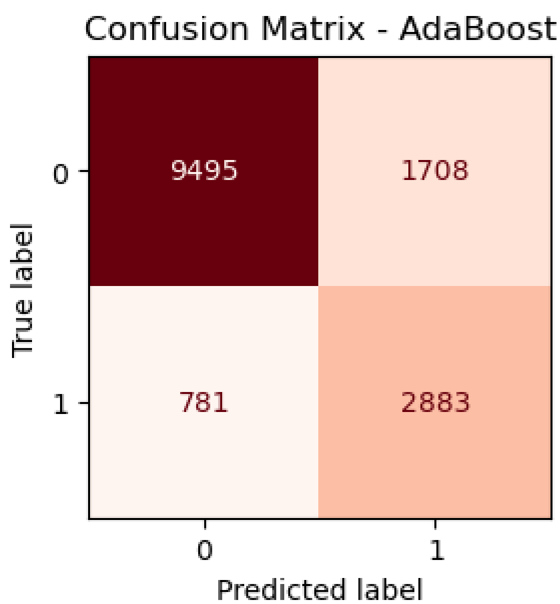
In [110…

```python
y_scores = grid_search_AdaBoost.predict_proba(X_test)[:, 1]
threshold = 0.499  # lower than 0.5, where it is centered
y_pred_adjusted = (y_scores >= threshold).astype(int)
```

In [111…

```python
# Compute confusion matrix
cm = confusion_matrix(y_test,
                      y_pred_adjusted
```

```
                    )

# Display confusion matrix as a heatmap
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=grid_search_AdaBoost.classes_
                             )
plt.figure(figsize=(3, 3)) #create a specific figure object in order to better mani

disp.plot(cmap="Reds",
          values_format='d', #show numbers as integers
          colorbar = False, #colorbar as legend disactivated
          ax=plt.gca() #plot inn the figure created
         )

plt.title("Confusion Matrix - AdaBoost")
plt.show()
```



In [112…

```
# Accuracy
acc = accuracy_score(y_test, y_pred_adjusted)
# Precision (by default, for positive class in binary classification)
prec = precision_score(y_test, y_pred_adjusted)
# Recall
rec = recall_score(y_test, y_pred_adjusted)
# F1-score
f1 = f1_score(y_test, y_pred_adjusted)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
Accuracy: 0.8326
Precision: 0.6280
Recall: 0.7868
F1-score: 0.6985
```

In [58]:
```python
# --- Saving ---
if recovered is False: #if the model currently used has not been recovered from ser
    with open("AdaBoost_loan.pkl", mode="wb") as f: #wb stands for Writing Binary
        cloudpickle.dump(grid_search_AdaBoost, f)

# --- Loading ---
"""
with open("AdaBoost_loan.pkl", mode="rb") as f: #rb stands for reading binary
    AdaBoost_recovered = cloudpickle.load(f)
"""
```

Out[58]:
```
'\nwith open("AdaBoost_loan.pkl", mode="rb") as f: #rb stands for reading binary\n
AdaBoost_recovered = cloudpickle.load(f)\n'
```

4.3. Gradient Boosting

In [59]:
```python
"""
Notice, if you do not have enough time, you can skip directly to the following cell
object in your folder, since it will open the already trained model.
"""

#setting the GradientBoosting model
gb_model = GradientBoostingClassifier(
                                    random_state=42
                                )

#adding to the pipeline the algorithm
full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', gb_model)
])

# Parameter grid for GridSearch
param_grid = {
    'classifier__n_estimators': [250, 350, 400],          #Number of boosting stage
    'classifier__learning_rate': [0.1, 0.2],       #Shrinkage (learning) rate
    'classifier__max_depth': [5, 6],                 #Maximum depth of decision tr
    'classifier__subsample': [0.8, 1.0],               #fraction of samples used
    'classifier__min_samples_split': [7, 10]           #minimum samples required
}

# Grid search setup
grid_search_GradientBoost = GridSearchCV(
                            full_pipeline,
                            param_grid,
                            cv=strat_kfold,
                            n_jobs=-1,
                            verbose=2,
                            scoring='recall',
                        )

start_time = time.time() #for counting the time
# Fit GridSearchCV on training data
grid_search_GradientBoost.fit(X_train,
```

```
                                y_train
                               )
end_time = time.time() #end time of execution
elapsed_time = convert_time(end_time - start_time)
print(f"GradientBoost training + grid search took {elapsed_time}")

# Best params and score
print("Best parameters found:", grid_search_GradientBoost.best_params_)
print("Best cross-validation Recall:", grid_search_GradientBoost.best_score_)

# Optional: Evaluate on the test set
test_score = grid_search_GradientBoost.score(X_test, y_test)
print(f"Test set Recall: {test_score:.4f}")
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits
GradientBoost training + grid search took 11:53:26
Best parameters found: {'classifier__learning_rate': 0.2, 'classifier__max_depth':
6, 'classifier__min_samples_split': 7, 'classifier__n_estimators': 400, 'classifier_
_subsample': 0.8}
Best cross-validation Recall: 0.6412736921910538
Test set Recall: 0.6638
```

In [60]:

```
#try to see whether the above cell has been run
try:
    grid_search_GradientBoost
    recovered = False #boolean variable to understand wheter the model has been rec

#otherwise open the serialized model
except:
    with open("GradientBoost_loan.pkl", mode="rb") as f: #rb stands for reading bin
        grid_search_GradientBoost = cloudpickle.load(f)
    recovered = True
```

In [61]:

```
pd.DataFrame(grid_search_GradientBoost.cv_results_)
```

Out[61]:

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__learning_
0	512.879887	144.450418	1.122826	0.063351	
1	529.986002	5.964861	0.987681	0.024942	
2	600.106107	3.549016	1.210050	0.031430	
3	739.277332	3.111717	1.232336	0.033994	
4	685.497165	2.432012	1.357316	0.045910	
5	847.365082	2.331266	1.351080	0.040197	
6	430.180005	2.764529	0.984268	0.015888	
7	517.996707	9.113573	1.025322	0.039395	
8	598.396039	1.338019	1.217375	0.010686	
9	735.358536	1.416284	1.214518	0.016678	
10	683.282903	0.908090	1.346113	0.030429	
11	844.259007	3.414119	1.331821	0.023504	
12	507.501319	0.807878	1.027414	0.021208	
13	616.086510	9.216419	1.039808	0.039840	
14	710.775291	1.752213	1.285066	0.008279	
15	884.544131	5.831086	1.295912	0.018324	
16	819.894631	7.279384	1.446876	0.050401	
17	1010.058585	7.963290	1.413886	0.011855	
18	509.259252	4.560607	1.040331	0.016265	

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__learning_
19	627.550477	3.987945	1.047841	0.016237	
20	711.698959	2.387991	1.296814	0.008935	
21	884.885003	2.905104	1.329976	0.066586	
22	821.041016	3.795591	1.429279	0.022201	
23	985.777877	14.838792	1.433254	0.032422	
24	434.285538	4.884939	1.011667	0.011140	
25	537.809077	1.623921	1.045920	0.017239	
26	611.762036	1.573480	1.265683	0.015563	
27	752.261048	1.697127	1.257984	0.009191	
28	698.959441	1.681796	1.611785	0.471492	
29	841.966967	9.355861	1.373461	0.010924	
30	438.581219	2.159170	1.023608	0.005995	
31	538.258345	0.487609	1.015282	0.015603	
32	613.360212	2.590131	1.257805	0.024218	
33	754.772213	3.014548	1.265249	0.016140	
34	694.920612	2.101590	1.362522	0.015444	
35	835.277452	13.329384	1.383765	0.037637	
36	518.270203	3.715741	1.067674	0.014647	
37	634.765384	1.988271	1.068276	0.022410	

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_classifier__learning_
38	714.320291	3.112515	1.312731	0.013647	
39	890.356776	3.208117	1.313919	0.030577	
40	818.291143	2.011892	1.456371	0.020871	
41	1017.280905	2.839392	1.437010	0.032551	
42	509.734762	1.242842	1.065272	0.015862	
43	632.832738	2.010486	1.059929	0.023055	
44	715.706904	3.373535	1.347079	0.043799	
45	866.709132	14.490509	1.293606	0.008553	
46	811.790967	4.671743	1.456968	0.036732	
47	918.061253	124.464338	1.118934	0.291508	

In [62]:

```
#we encapsulate just the best model
GradientBoost_whole_model = grid_search_GradientBoost.best_estimator_ #here we save
GradientBoost_classifier = GradientBoost_whole_model.named_steps['classifier'] #her
```

In [63]:

```
#recover just the preprocessor from the whole model
preprocessor = GradientBoost_whole_model.named_steps['preprocessor']
feature_names = []

for name, transformer, cols in preprocessor.transformers_:
    #if the transformer is a drop operator
    if transformer == 'drop':
        continue
    #if the transformer is actually a pipeline object, so that has inside other tra
    if isinstance(transformer, Pipeline):
        #get the last step of the pipeline, because the last step is the one that f
        last_step = transformer.steps[-1][1]
        if hasattr(last_step, 'get_feature_names_out'):
            names = last_step.get_feature_names_out(cols)
        else:
            names = cols
    #if the transformer is directly a transformer
    else:
        if hasattr(transformer, 'get_feature_names_out'):
            names = transformer.get_feature_names_out(cols)
        else:
```

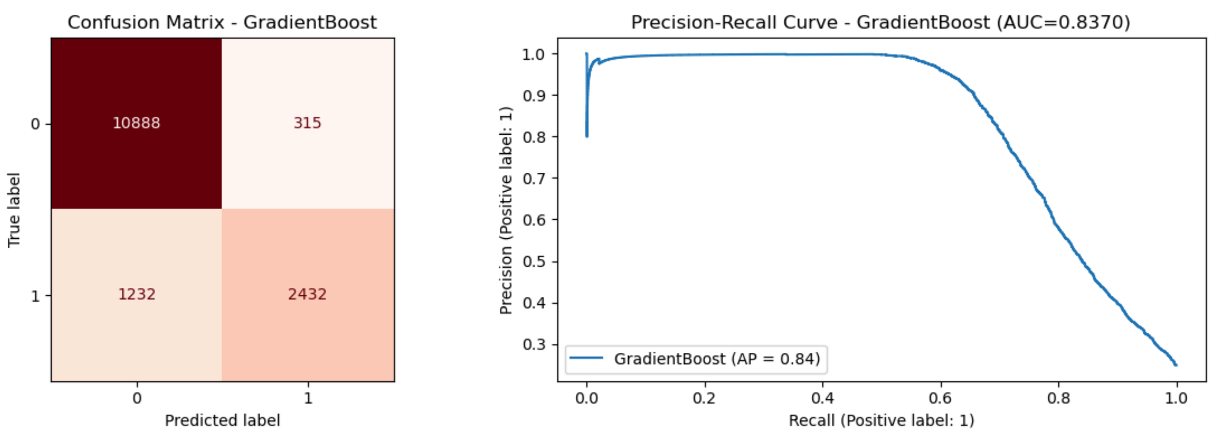
```
            names = cols

    feature_names.extend(names)

feat_imp = pd.DataFrame({
                        'feature': feature_names,
                        'importance': GradientBoost_classifier.feature_importances_
                        }).sort_values(
                                        by="importance",
                                        ascending=False
                                        )

print(feat_imp)
```

```
                                feature  importance
43                       property_value    0.306946
46                                  LTV    0.254441
47                                dtir1    0.080428
44                               income    0.063137
45                         Credit_Score    0.031656
34                      credit_type_CIB    0.028567
41                          loan_amount    0.026394
24                      loan_type_type1    0.023346
12                lump_sum_payment_lpsm    0.015999
8              Neg_ammortization_neg_amm    0.013691
26                      loan_type_type3    0.011565
48                                  age    0.009984
13            lump_sum_payment_not_lpsm    0.009810
27                      loan_purpose_p1    0.009537
9             Neg_ammortization_not_neg    0.009407
4                  Credit_Worthiness_l1    0.009114
19    submission_of_application_to_inst    0.007774
42                                 term    0.006949
3                      approv_in_adv_pre    0.006822
25                      loan_type_type2    0.006762
29                      loan_purpose_p3    0.006734
18   submission_of_application_not_inst    0.005684
31                    occupancy_type_ir    0.005153
30                      loan_purpose_p4    0.004846
5                  Credit_Worthiness_l2    0.003642
0                         loan_limit_cf    0.003250
1                        loan_limit_ncf    0.002686
2                    approv_in_adv_nopre    0.002615
49                          total_units    0.002610
32                    occupancy_type_pr    0.002479
39                       Region_central    0.002139
37                         Region_North    0.002114
28                      loan_purpose_p2    0.002072
21                         Gender_Joint    0.002005
22                          Gender_Male    0.001984
16         co-applicant_credit_type_CIB    0.001968
23             Gender_Sex Not Available    0.001961
35                     credit_type_CRIF    0.001826
10               interest_only_int_only    0.001752
36                      credit_type_EXP    0.001739
17         co-applicant_credit_type_EXP    0.001429
20                        Gender_Female    0.001421
11                interest_only_not_int    0.001183
33                    occupancy_type_sr    0.001170
40                         Region_south    0.001095
38                    Region_North-East    0.000806
14                 construction_type_mh    0.000474
15                 construction_type_sb    0.000463
7                       open_credit_opc    0.000189
6                      open_credit_nopc    0.000183
```

In [64]:

```python
# Predict on the test set
y_pred = grid_search_GradientBoost.predict(X_test)

# Compute confusion matrix
```

```python
cm = confusion_matrix(y_test, y_pred)

# Get predicted probabilities for the positive class
y_scores = grid_search_GradientBoost.predict_proba(X_test)[:, 1]

# Compute average precision (AUC-PR)
auc_pr = average_precision_score(y_test, y_scores)

# Create a figure
fig, axes = plt.subplots(1, # one row
                         2, # two columns
                         figsize=(12, 4)
                        )

# --- Confusion Matrix ---
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=grid_search_GradientBoost.classes_
                             )
disp.plot(cmap="Reds",
          values_format='d',
          colorbar=False, #don't show the legend colormap
          ax=axes[0])
axes[0].set_title("Confusion Matrix - GradientBoost")

# --- Precision-Recall Curve ---
PrecisionRecallDisplay.from_predictions(y_test,
                                        y_scores,
                                        name="GradientBoost",
                                        ax=axes[1]
                                       )
axes[1].set_title(f"Precision-Recall Curve - GradientBoost (AUC={auc_pr:.4f})")

plt.tight_layout()
plt.show()
```

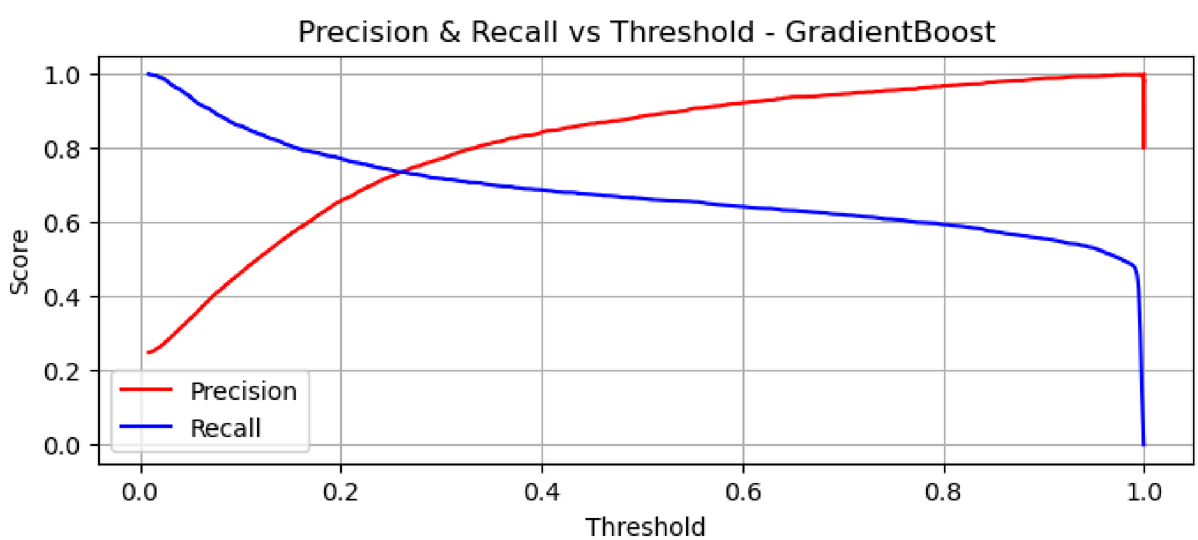


In [65]:
```python
# Predict on test set
y_pred = grid_search_GradientBoost.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)
# Precision (by default, for positive class in binary classification)
prec = precision_score(y_test, y_pred)
```

```python
# Recall
rec = recall_score(y_test, y_pred)
# F1-score
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
Accuracy: 0.8959
Precision: 0.8853
Recall: 0.6638
F1-score: 0.7587
```

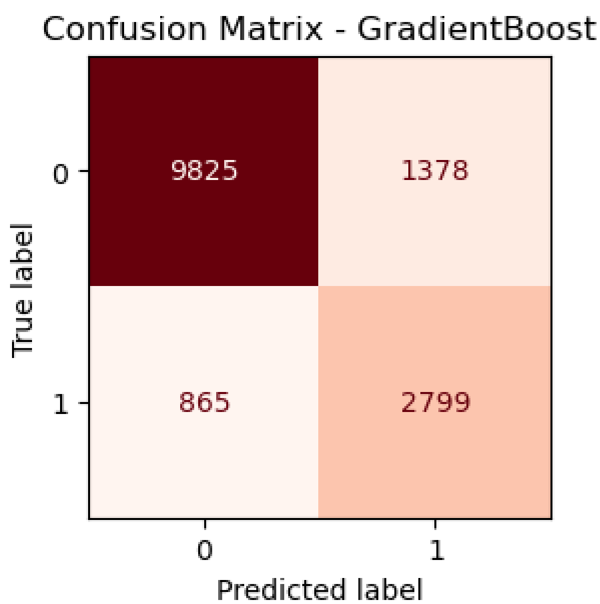
In [66]:
```python
# Get predicted probabilities for the positive class
y_scores = grid_search_GradientBoost.predict_proba(X_test)[:, 1]

# Compute precision, recall, thresholds
precision, recall, thresholds = precision_recall_curve(y_test, y_scores)

# Plot Precision and Recall vs Threshold
plt.figure(figsize=(8, 3))
plt.plot(thresholds, precision[:-1], label='Precision', color='red')
plt.plot(thresholds, recall[:-1], label='Recall', color='blue')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.title('Precision & Recall vs Threshold - GradientBoost')
plt.legend()
plt.grid(True)
plt.show()
```



In [123…
```python
y_scores = grid_search_GradientBoost.predict_proba(X_test)[:, 1]
threshold = 0.21  # lower than 0.5, where it is centered
y_pred_adjusted = (y_scores >= threshold).astype(int)
```

In [124…
```python
# Compute confusion matrix
cm = confusion_matrix(y_test,
                      y_pred_adjusted
```

```
                      )

# Display confusion matrix as a heatmap
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=grid_search_GradientBoost.classes_
                             )
plt.figure(figsize=(3, 3)) #create a specific figure object in order to better mani

disp.plot(cmap="Reds",
          values_format='d', #show numbers as integers
          colorbar = False, #colorbar as legend disactivated
          ax=plt.gca() #plot inn the figure created
         )

plt.title("Confusion Matrix - GradientBoost")
plt.show()
```



In [125…

```
# Accuracy
acc = accuracy_score(y_test, y_pred_adjusted)
# Precision (by default, for positive class in binary classification)
prec = precision_score(y_test, y_pred_adjusted)
# Recall
rec = recall_score(y_test, y_pred_adjusted)
# F1-score
f1 = f1_score(y_test, y_pred_adjusted)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1-score: {f1:.4f}")
```

```
Accuracy: 0.8491
Precision: 0.6701
Recall: 0.7639
F1-score: 0.7139
```

In [70]:
```python
# --- Saving ---
if recovered is False: #if the model currently used has not been recovered from ser
    with open("GradientBoost_loan.pkl", mode="wb") as f: #wb stands for Writing Bin
        cloudpickle.dump(grid_search_GradientBoost, f)

# --- Loading ---
"""
with open("GradientBoost_loan.pkl", mode="rb") as f: #rb stands for reading binary
    GradientBoost_recovered = cloudpickle.load(f)
"""
```

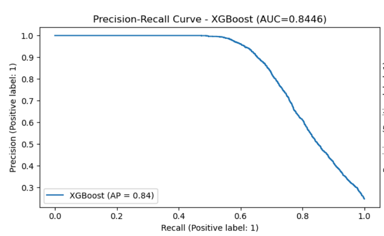
Out[70]:
```
'\nwith open("GradientBoost_loan.pkl", mode="rb") as f: #rb stands for reading bin
ary\n    GradientBoost_recovered = cloudpickle.load(f)\n'
```

5. Comparisons

Now that all three models have been run we can make comparisons and draw conclusions.

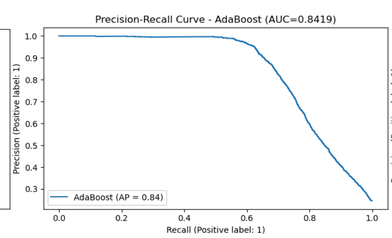
Since in our dataset the positive class is rare, as a rule of thumb we should prefer the *Precision-Recall curves* for comparisons, hence we will combine them with the best scores found after tweaking the precision-recall threshold, in order to choose the best model.

XGBoost



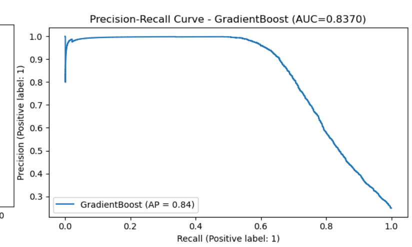
Accuracy: 0.8255
Precision: 0.6117
Recall: 0.7997
F1-score: 0.6932

AdaBoost



Accuracy: 0.8326
Precision: 0.6280
Recall: 0.7868
F1-score: 0.6985

GradientBoost



Accuracy: 0.8491
Precision: 0.6701
Recall: 0.7639
F1-score: 0.7139

See Appendix 7.B. to understand how these images have been generated

All three models are quite good, and have similar scores.
On the one hand XGBoost and GradientBoost have different scores each one with its own pros and cons, while AdaBoost seems the the model in the middle.
If we take into account the overall F1-score that is basically the harmonic mean between precision and recall:

$$ \text{F1-score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} \ + \ \text{recall}} $$

We would choose GradientBoost, since this metric is comprehensive of both Precision and Recall.

If we want to dig deeper into the curves, the shape of the PR-curve for GradientBoosting has a strange short drop at the very beginning, when the threshold is set at nearly its maximum. This phenomena could derive from several reasons, but mathematically speaking it depends on the fact that when we set the threshold at very high values, the number of samples captured is very low, so the `precision` can encompass only few true positive samples, so if the model misclassifies even a very low amount of negative instances, the metric goes down all of a sudden.
This is supported by the fact that our dataset, as previoiusly shown, has a low amount of postive instances compared to negative ones.
Furthermore, the Precision & Recall Vs Threshold, curve for GradientBoosting and XGBoosting, demonstrate this artefact, because when the threshold is nearly at its maximum, in these two cases, the Recall drops all of a sudden.
Another possible explanation for the unusual behaviour of GradientBoost's PR-curve is that unlike the other two models, GradientBoost does not have a straightforward parameter to compensate an unbalanced dataset; we can do it, but in a longer and a more manual setup (*e.g.:* via sample weights). (Notice that for XGboost and AdaBoost, we set the models at the beginning with balancing out the dataset).
In any case, this is a common behaviour when dealing with unbalanced datasets, and does not necessarily mean that a model is weak, instead GradientBoost is again quite good, as the Area Under the Curve witnesses.

On the other hand if we take into account the general principle of this project, we want to prioritize recall over precision and even further, if we take a look at training time:

XGBoost	AdaBoost	GradientBoost
00:16:58	04:45:10	11:53:26

XGBoost is the one that performs better
Even under the perspective of the Area Under the Curve, that for comparisons is one of the main metrics that is taken under consideration, as it gives a numerical summary of a model's overall performance. Particularly useful for imbalanced datasets. It represents the total area enclosed by the PR-curve, which plots precision against recall across different classification thresholds. A higher PR AUC value, closer to 1, indicates a better-performing model, showing a superior ability to correctly identify positive cases while minimizing false positives.
All this is due to some intrinsic characteristics of the implementation of the algorithm itself. XGBoost is built upon the core Gradient Boosting framework, but it has some key features such as smarter penalizations and regularization, adopts Newton Boosting, furthermore XGBoost can build parts of the model in parallel. This not only speeds up the training process but also makes it scalable, allowing it to handle very large datasets by distributing

the workload across multiple machines or processing units.

Therefore, although GradientBoost achieves the best F1 and Accuracy, XGBoost provides the best trade-off between Recall, overall PR AUC, and computational efficiency, making it the most suitable choice for our project.

Notice: if your model takes a lot of time to train (e.g.: 24h+), make sure to periodically save the jupyter notebook file, because the Cross-site request forgery token (`XSRF`) may get out fo sync, leading to a restart of the page without saving the variables! see

6. Conclusion and Possible Expansions

A simple and very effective strategy that we can implement in order to enhance quickly our models can be achieved by comparing the best parameters found for each model with the ranges that we have set in the `param_grid` dictionaries.
By doing so, we can tweak the ranges accordingly and re-run the models many times until we reach the optimal solutions with the proper ranges.
With a similar approach we can even expand the *hyperparameters* to tweak, since there exist many more, of course this will lead to slow down the process. A possible way to cope with this latter issue is by using `RandomizedSearchCV`, it evaluates a given number of random combinations by selecting a random value for each hyperparameter at every iteration.

This project can be further improved under some aspects that can be seen as a good starting point for future directions.
One of the first improvement that can come up in mind straight away after analyzing section 4. is the further feature selection, since all three algorithms under the hood of features importance show that: `open_credit_opc` , `open_credit_nopc` and `construction_type_sb` have no importance in reducing impurity.
Plotting the **learning curves** and eventually implementing some strategies such as **early stopping** can be very beneficial especially for Adaptive Boosting and Gradient Boosting, since they take long time for training.
Since the dataset results a bit unbalanced towards the negative samples, we could implement **SMOTE** *(Synthetic Minority Over-sampling Technique)* techniques in order to generate synthetic samples from the minority class, so to compensate it.
Furthermore we can try to apply dimensionality reduction and/or different models, maybe even approaching the field of Artificial Neural Networks, starting with the base model of the Multilayer Perceptron.

The main purpose of this notebook has originally started as a Kaggle challenge, it can be actually seen a good starting point for many other interesting applications, not necessarily

related to the credit industry.
For example with the same binary classifier concept we can implement fraud dectors, spam email dector, malicious cyber attacks identifier and so on...

7. Appendix

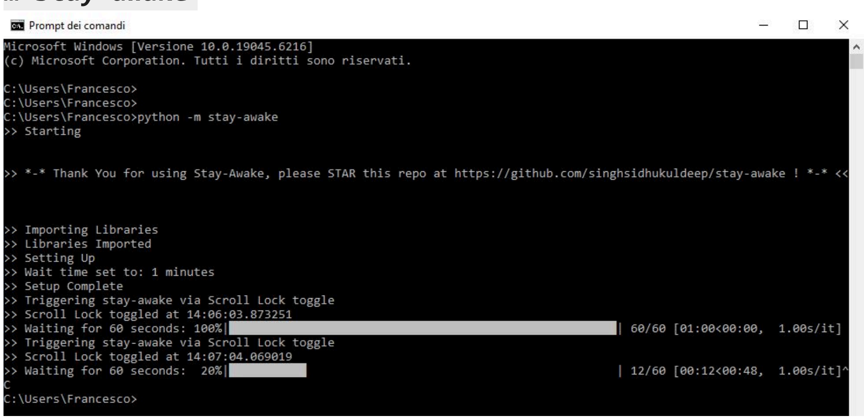
This section contains some tips and interesting tricks that I have used during the execution of this Notebook.

7.A. Python stay-awake module

At the end of section 5. we higlighted the training time for the three models. If we sum all training times we quickly understand that in total, running this notebook from scratch requires approximately seventeen hours.
If we do not want to change the settings of our computer in order to prevent sleep mode, we can use a Python script that runs in a different thread as a background daemon, since it requires very few CPU.
Online there are many modules from which we can take advantage of.

For example this one here is very easy and straighforward to implement, even though if you run it on Windows 10 or newer versions you need to adjust it a bit, because automatic mouse moving does not prevent sleep mode anymore see.
Thus, once you have found out the location of the script in your computer with: `pip show stay-awake` you can change the `__init__.py` inside the `stay-wake` folder script so that instead of moving the mouse it activates/deactives the ScrollLock button.

Once the script is ready, it is very easy to use it, just open a Shell prompt ad type: `python -m stay-awake`



See Appendix 7.B. to understand how this image has been generated

7.B. Python image converter base64 (PDF exporting)

If we want to convert a file Jupyter Notebook with images into PDF format, all the media attachments will be lost, this is a well-known bug of `ipynb` files.
In order to overcome this issue there a few tricks, the one I have implemented is based on the Python library `base64`. Basically it converts images into ASCII characters, by opening the attachment in binary mode and then encoding it in series of printable characters limited to a set of 64 unique characters. More specifically, the source binary data is taken 6 bits at a time, then this group of 6 bits is mapped to one of 64 unique characters.

Base64 encoding causes an overhead of 33–37% relative to the size of the original binary data (33% by the encoding itself; up to 4% more by the inserted line breaks).

For further explanations see: GitHub issue, Base64

The following block of codes are the ones that generated the encoding and stored directly the content into `HTML` tags for the images we saw through this notebook earlier.
Of course, once the attachment has been encoded, we can remove it from the directory.

For the sake of brevity we do not show the output of the blocks.

Each block is led by the `Markdown` referenced cell, that we would have written in case we had not had any issues.

In [ ]:
```
<img src="loan.JPG" style="width:75%;">
```

In [137…
```
import base64 #library for base64 encoding

file = "loan.JPG"

with open(file, "rb") as f:
    data = f.read()

encoded = base64.b64encode(data).decode("utf-8")

# Genera il tag HTML con base64
html_code = f'<img src="data:image/jpeg;base64,{encoded}" style="width:75%;">'

#print(html_code)
```

In [ ]:
```
<div style="display: flex;
            justify-content: center;
            text-align: center;">

  <div>
     XGBoost
    <img src="PR-curve XGBoost.png" style="padding:1%"><br>
    Accuracy: 0.8300 <br>
    Precision: 0.6105 <br>
    Recall: 0.7780 <br>
    F1-score: 0.6842
  </div>
```

```
  <div>
      AdaBoost
    <img src="PR-curve AdaBoost.png" style="padding:1%"><br>
    Accuracy: 0.8247 <br>
    Precision: 0.6020 <br>
    Recall: 0.7652 <br>
    F1-score: 0.6738
  </div>

  <div>
      GradientBoost
    <img src="PR-curve GradientBoost.png" style="padding:1%"><br>
    Accuracy: 0.8444 <br>
    Precision: 0.6470 <br>
    Recall: 0.7533 <br>
    F1-score: 0.6961
  </div>

</div>
```

In [134…

```
files = ["PR-curve XGBoost.png",
         "PR-curve AdaBoost.png",
         "PR-curve GradientBoost.png",
        ]
labels = ["XGBoost",
          "AdaBoost",
          "GradientBoost",
         ]
accuracy = [0.8255,
            0.8326,
            0.8491,
           ]
precision = [0.6117,
             0.6280,
             0.6701,
            ]
recall = [0.7997,
          0.7868,
          0.7639,
         ]
f1 = [0.6932,
      0.6985,
      0.7139,
     ]

html = '<div style="display: flex; justify-content: center; text-align: center;">\n

for i in range(len(files)):
    with open(files[i], "rb") as f:
        data = f.read()
    encoded = base64.b64encode(data).decode("utf-8")

    html += f'''  <div>
      {labels[i]}
      <img src="data:image/png;base64,{encoded}" style="padding:1%"><br>
```

```
      Accuracy: {accuracy[i]:.4f} <br>
      Precision: {precision[i]:.4f} <br>
      Recall: {recall[i]:.4f} <br>
      F1-score: {f1[i]:.4f}
  </div>\n\n'''

html += '</div>'

#print(html)
```

In [ ]:

```
<img src="stay-awake.JPG" style="width:75%;">
```

In [73]:

```
file = "stay-awake.JPG"

with open(file, "rb") as f:
    data = f.read()

encoded = base64.b64encode(data).decode("utf-8")

html_code = f'<img src="data:image/jpeg;base64,{encoded}" style="width:75%;">'

# print(html_code)
```